IE 7945—Masters Project
Group A: Edward Brown, Atera Alam, William Hall, Murari Guragain, Peter Azmy
Milestone #4-Project Showcase

Abstract

Over 16,000 articles between the years 1990 and 2004 were analyzed to uncover trends of changes in suicide research from the lens of focus on suicide research over time, research methodologies, author sentiment, and possible suicide prevention strategies based on article content. Data for this research project was scraped from SafetyLit as required by the stakeholder. Some required data, such as grant information, DOI and keywords, were missing, for which additional websites, such as PubMed, were scrapped. Additionally, to fill in the remainder of missing information, such as suicide prevention methods and research methodologies, Natural Language Processing (NLP) techniques were used. From authorship analysis, it was found that the majority of authors contributed to single publications, whereas a small number of authors published repeatedly. Through temporal analysis, it was found that, over the course of the years, research methodologies oriented more towards quantitative techniques. Number of overall publications on suicide also increased throughout the years. Number of publications in prominent journals over the years was also analyzed.
Through sentiment analysis, it was found that the major emotions detected in this research were sadness and fear **(Figure F)**. Contributions of prolific authors were also investigated and compared to the overall suicide research sum of contributions.

Introduction

The purpose of this project is to identify changes in research on suicide. To do this, our group had scraped data from 1990 to 2004 from SafetyLit. SafetyLit is a website that contains data on injuries and accidents.
Suicide is a public health issue and its grasp has only widened in the past few decades in the United States and in the world. Due to the rise in suicide, the amount of research has also grown over the past few decades. Many prominent medical and psychological journals have published studies on suicide since 1990. Understanding how suicide research changed over the course of the past 35 years can be critical in identifying key information, such as the biggest author and journal contributors.
This study provides an analysis of suicide research over the period of 1990 to 2004. We collected data from SafetyLit, a website curated for accidents and injuries. The dataset contains 16,075 articles, which helped us in uncovering key patterns and developing NLP techniques to uncover connections and additional information.

We used the collected data to derive insights such as temporal trends in suicide research publication, author trends in suicide research, and trends in research methodology. Furthermore, we used NLP techniques to drive sentiment analysis of suicide research and detected emotions of the articles based on textual nuances.
This research is intended to shed light on patterns, changes and driving factors of suicide research between 1990 and 2004.
Beyond merely tracking publication volume, this study also investigates the evolving language and framing used in suicide research, which may reveal shifting societal attitudes, funding priorities, and methodological rigor over time. Understanding these changes is essential for researchers, mental health professionals, and policymakers alike, as it helps identify trends in scholarly attention, under-researched subtopics, and opportunities for cross-disciplinary collaboration. Our work bridges computational techniques with public health insights by leveraging web scraping, natural language processing (NLP), and sentiment analysis to uncover latent structures and emotional undercurrents in the academic literature. By systematically examining thousands of articles published between 1990 and 2004, we aim to create a foundation for future longitudinal studies in mental health scholarship and provide a reproducible methodology for similar meta-research endeavors.

Methodology

Data Collection: To facilitate the collection of data, we built a robust scraper that collected the required data from the stakeholders. The required data included author details, author affiliations, journal names, publication year, language, keywords, DOI, research methodology, data source, grant information, suggested suicide prevention measures, and any other information such as abstracts.
Through multiple rounds of scrapes, we collected the following data: URL, DOI, Data Source, ISBN, ISSN, Language, Abstract, Research Methodology, Keywords, Article Title, Book Title, Pages, Volume, Publication year, Journal name, series, publisher, edition, editor and grant information.
Initially we scraped SafetyLit manually, but the manual process was unable to scrape for targeted information. Then we built a scraper for the required fields: author details, author affiliations, journal names, publication year, language, keywords, DOI, research methodology, data source, grant information, suggested suicide prevention measures, and any other information such as abstracts (appendix 1). The scraper was able to collect **16,158** articles, as represented in the v1.raw_scrape file. The scraper could not find multiple required fields, such as keywords, research methodology, and grant information. Most DOIs were missing as well. To address these issues, subsequent scraper and NLP methodologies were used to scrape information or to impute data. To collect information on grant information, the official pubmed website was used. To collect keywords, the official pubmed was used as well. To find DOIs for the articles, a pubmed scrape was conducted as well (appendix).

Data Validation: For data validation processes, whitespaces were removed, exact duplicates were dropped, publication years were verified to be within the 1990-2004 range, and obvious parsing errors such as truncated texts or misaligned fields were corrected. HTML notations were removed by going through the entire dataset manually. Then, 100 entries were selected at random and cross-checked with the search results to ensure that the final dataset was accurate. The author names were also separated to distinguish first authors as well. After data validation processes, we reached a final number of 16,075 articles with the maximum information we could collect through previously mentioned methods.

NLP Methods to Impute Results:
Suicide Prevention Measures: As suicide prevention measures were not present in the available data, NLP methodologies were used to impute them. To do so, we developed a text-mining script to scan abstracts and retrieve the top 20 keywords of those abstracts through utilizing a TF-IDF Vectorizer. Then, those top keywords were used to match them with different types of suicide prevention methods, namely, medication, crisis intervention and support services, psychotherapy, etc.
Research Methodology: Beginning with lists of commonly used words and phrases related to each methodology (quantitative and qualitative), synonyms for each were found with the WordNet feature of NLTK and appended to their respective lists. Each article's abstract was then scraped for mentions of any of these terms. Each mention was added to a score for each respective methodology, and the greater of the two scores determined the given label. For articles entirely missing abstracts or that contain an equal amount of qualitative and quantitative keywords, a label of "Mixed Methods or Unclear" was assigned. All of the analyses were carried out using Python, and results were visualized for interpretation.

Database: The collected data are stored in Azure. Our database consisted of 5 tables, which are detailed below.

1. dbo.Articles_Final_Cleaned_v2

Purpose: Stores comprehensive metadata about research articles.

Key Columns:

- ID (PK): Unique identifier for each article.
- Title, URL, Publication_Year, Language, Abstract: Core descriptive fields.
- EntryType, Pages, Volume, Number: Bibliographic formatting data.
- Research_Methodology, Data_Source: Academic metadata.
- DOI: Digital Object Identifier.
- Journal_Name: Name of the publishing journal (used for linking to Journals_Final_Cleaned_v2).

2. dbo.Authors_Table_With_All_Fields

Purpose: Captures authorship information for articles.

Key Columns:

- ID: Corresponds to the article (foreign key to Articles_Final_Cleaned_v2).
- Author_Affiliations: Institutional affiliations of authors.
- first_author, author_2, ..., author_42: Up to 42 authors per article.

3. dbo.Keywords_Final_Cleaned_v2

Purpose: Contains keywords associated with each article.

Key Columns:

- ID: Foreign key corresponding to the article.
- Keywords: List of relevant keywords (could be a comma-separated string or a list).

4. dbo.Journals_Final_Cleaned_v2

Purpose: Stores metadata about journals.

Key Columns:

- ID: Unique journal identifier.
- Journal_Name: Title of the journal.
- ISSN, ISBN: Standard identifiers for print/digital journals.
- Publisher: Publishing entity.

5. dbo.Grants_Final_Cleaned_v2

Purpose: Records grant/funding information for each article.

Key Columns:

- ID: Foreign key corresponding to the article.
- Grant_Information: Details of funding sources, grant numbers, agencies, etc.

Relationships Overview

- ID is the primary linking field across tables.
- Journal_Name in Articles_Final_Cleaned_v2 can be joined with Journals_Final_Cleaned_v2 for journal metadata.
- Authors, Keywords, and Grants are auxiliary tables supporting the main Articles table.

Results & Discussion

Data Overview: From the scrape and NLP imputations, we scraped a total of **16,075** published articles. We also scraped **1474** grant information and **7574** keywords through scraping PubMed. Through using NLP techniques, we imputed **3025** suicide prevention measures.

Research Trend:
Here are some trends we have noticed through this research:

Increase in Publication: Research articles on suicide had increased significantly. In 1990, around 1000 publications were recorded, and in 2004 the research publications were above 1500, which is ~50% increase in publications, as seen in Figure A below.

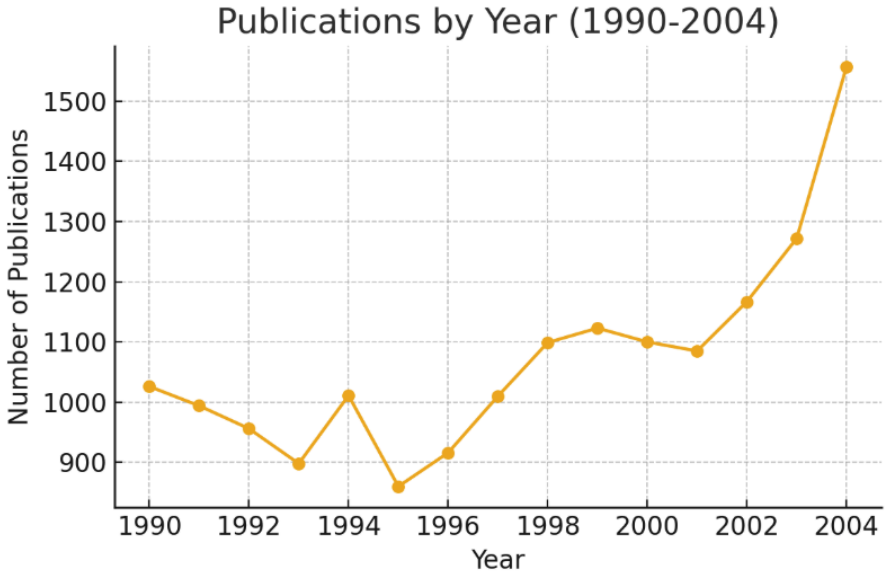

Figure A

Shift in Research Methodology: Through understanding research methodologies, we also see a shift. In the early 1990s, we see most of the studies being “mixed methods or unclear” in approach, with very few articles being labeled quantitative, more specifically 13%. Wherein by the early 2000s, quantitative research reached about 25% and mixed or unclear methodologies dropped to about 60%, as shown in Figure B below:

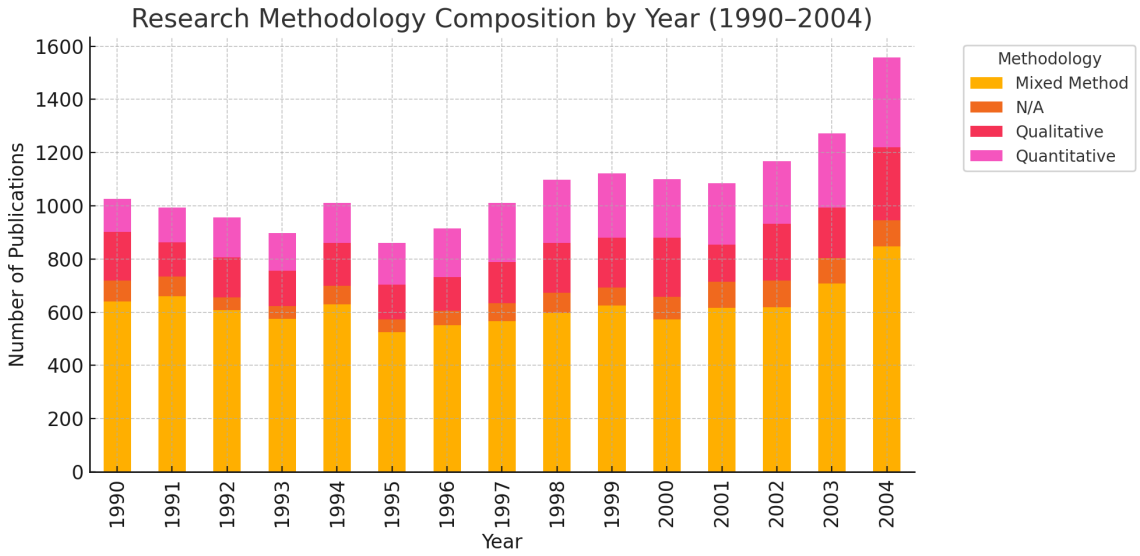

Figure B

<u>Publications of Suicide in Key Journals:</u>

Key journals across medicine, psychiatry and suicide-specialized journals such as the American Journal of Psychiatry, Archives of Suicide Research, British Journal of Psychiatry, BMJ, Crisis, JAMA, Journal of Affective Disorders, Lancet, Psychological Reports and Suicide and Life-Threatening Behavior published around the same number of articles each year. However, the numbers had gone up in the last 2 years of our research focus, 2003-2004. This can be seen in Figure C below.

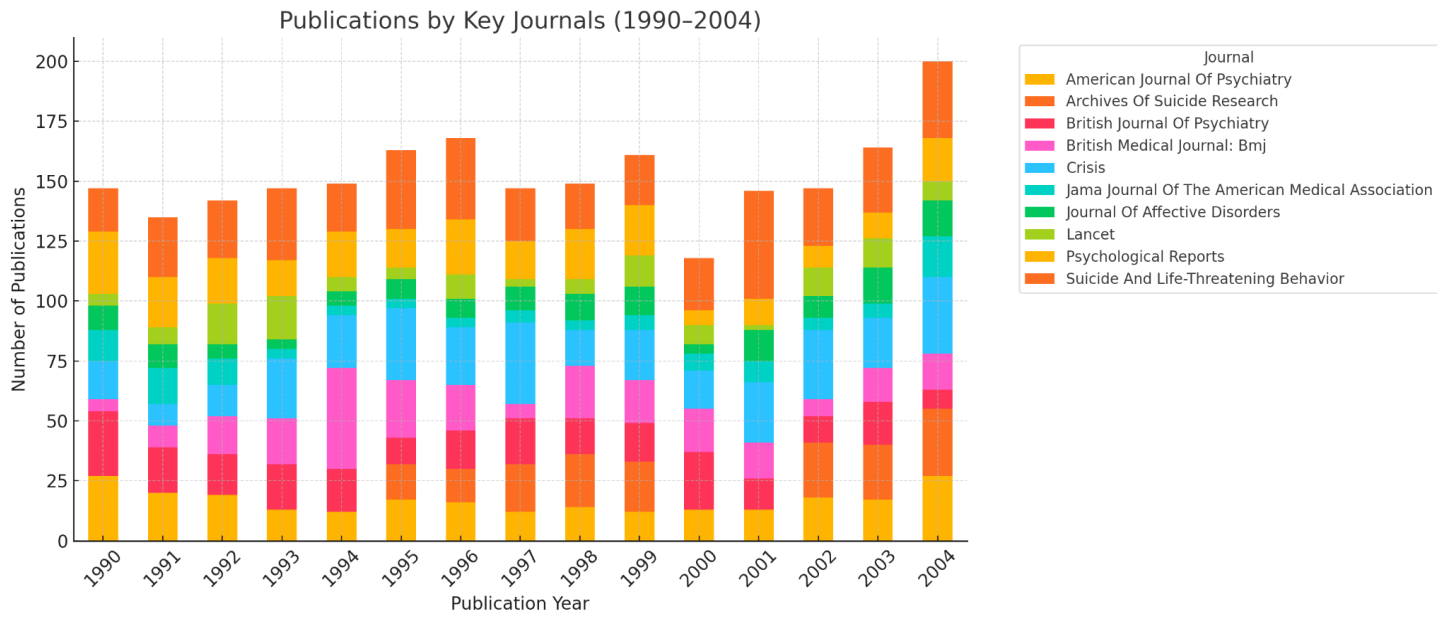

Figure C

<u>Data Modeling:</u>

<u>Approach:</u> To derive meaningful patterns from the curated corpus, we employed a multi-faceted data modeling strategy combining author classification, emotion detection, sentiment analysis, and co-occurrence network analysis. Recognizing the importance of scholarly engagement levels, we first categorized authors based on their publication frequency, distinguishing prolific researchers from less frequent contributors. This allowed us to contrast linguistic patterns, emotional tone, and research focus across experience levels.

For sentiment and emotion analysis, we applied pre-trained transformer-based NLP models (e.g., VADER and TextBlob for sentiment; NRC lexicon for emotion tagging) to the abstracts and, when available, full-text summaries. These models enabled us to extract fine-grained affective signals, which we then aggregated by author type and journal to identify broader stylistic and thematic trends.

In addition to individual-level analyses, we constructed co-authorship networks using PyVis and NetworkX to visualize collaboration patterns and identify central hubs in the academic ecosystem. This network modeling approach provided insight into research clusters and interdisciplinary intersections, shedding light on how collaborative structures may influence research output and tone.

Finally, we used dimensionality reduction techniques (e.g., PCA) and clustering algorithms to explore thematic groupings in suicide prevention keywords, offering a macroscopic view of topical emphases over time. These modeling strategies, taken together, enabled a comprehensive, data-driven narrative about the evolution of suicide research across a 15-year span.

Prolific Authors:
In order to better analyze patterns amongst authors, we elected to split authors into two groups; Prolific & Non-Prolific. Prolific authors are defined by us as having published 3 or more papers during the year of our analysis. We wanted to analyze differences in emotions and sentiments between authors who wrote extensively on the subject of suicide and those who were less involved in researching suicide.

Top 10 Most Prolific Authors:

1	Pompili, Maurizio	11
2	Girardi, Paolo	11
3	Tatarelli, Roberto	10
4	Lester, David	8
5	Lester, D.	7
6	Mann, J. John	6
7	Ruberto, Amedeo	6
8	Suominen, Kirsi	5
9	Goldney, Robert D.	5
10	Kessing, Lars Vedel	5

Figure D

Figure D shows that the most prolific of authors published an extraordinarily high volume of research in 2004. With three separate authors being published 10 or more times, and 2 authors being published an astonishing 11 times. A mere 10 authors accounted for 74 papers. With a total article count of 1,566, these 10 authors account for 4.7 percent of our total suicide research for the year.

What was interesting to see was that, even though these authors individually published a lot over the 15 years, no single author drove the field. Even though these prolific authors contributed to suicide research, the majority of the research contributions were from authors who published occasionally. This can be good for suicide research, as there is no one dominating voice or thought member; the ideas, research findings and contributions are coming

from different backgrounds, bringing richness to suicide research. A visualization is added below in Figure E.

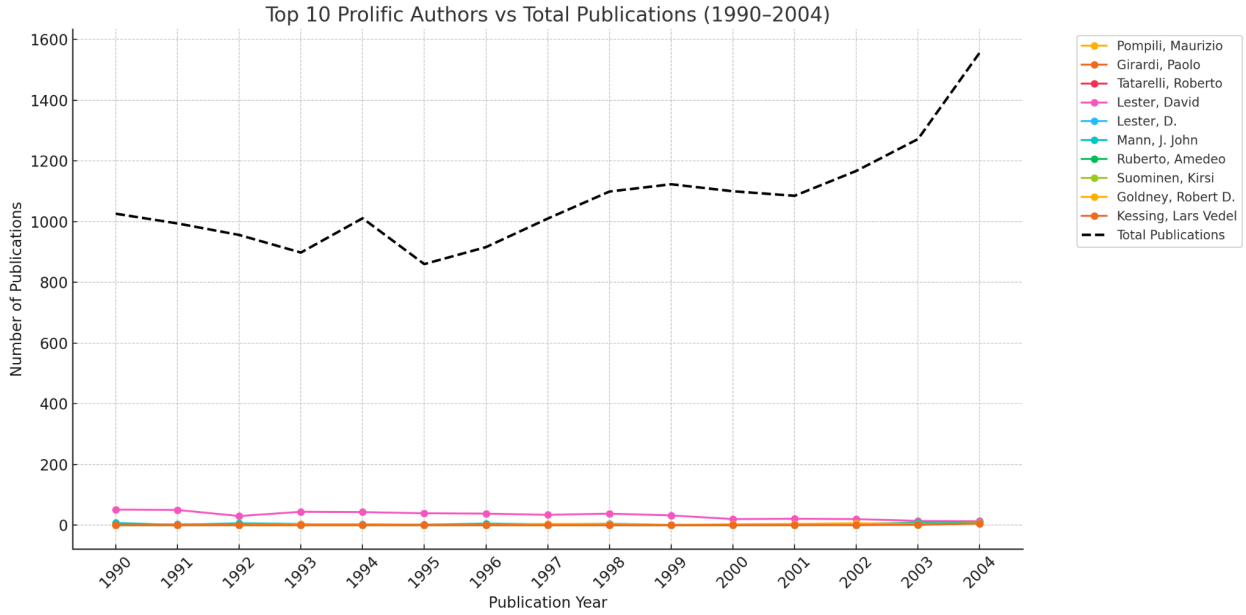

Figure E

<u>Emotion Detection Amongst Prolific Authors in the Year 2004:</u>

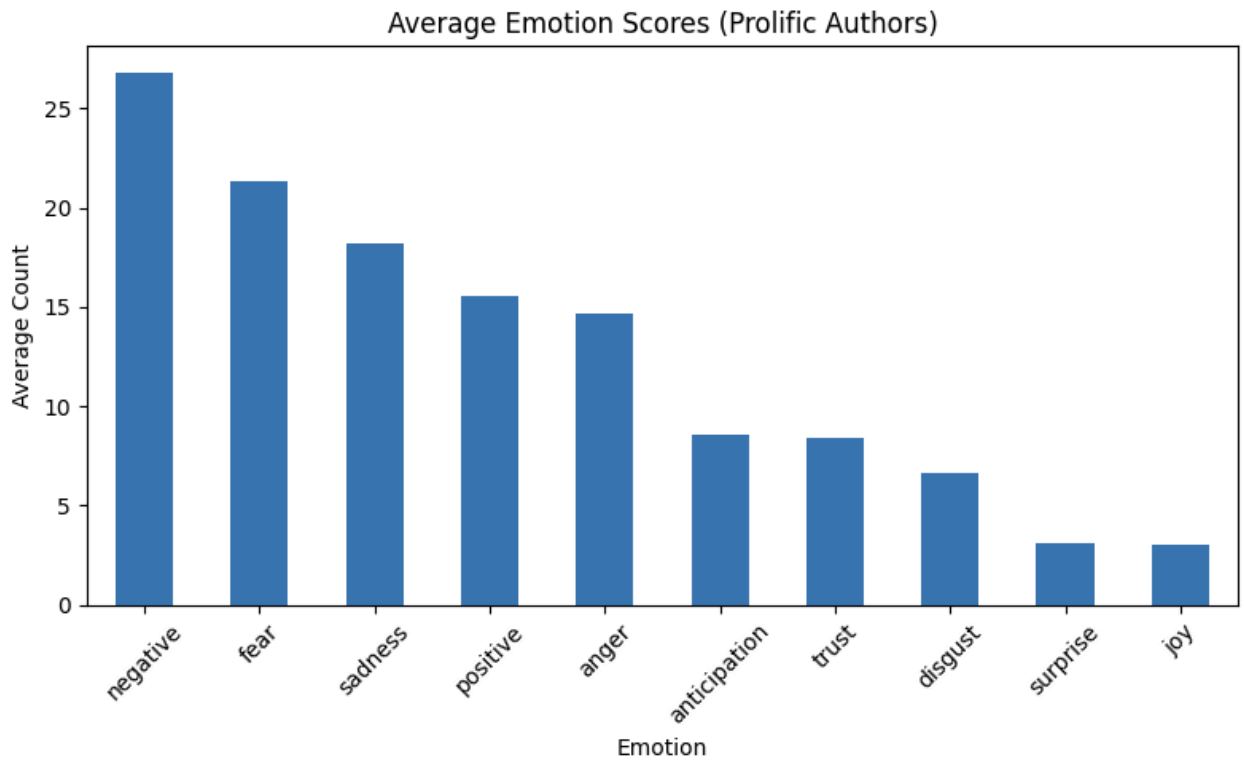

Figure F

Figure F presents the average emotion scores derived from text authored by researchers who published three or more suicide-related papers in the year 2004. The analysis employs an emotion detection model to quantify emotional tone based on the presence of specific affective terms within the body of research articles. Emotions are categorized according to a standard lexicon, including both positive and negative affective states.

The most prominent emotional categories among prolific authors are overwhelmingly negative in nature. The category negative registers the highest average count, followed closely by fear and sadness, suggesting a dominant negative emotional framing in the discourse. While positive sentiment appears in a moderate range, its average score is notably lower than that of the top three negative emotions. Anger also appears frequently, underscoring the affective intensity associated with the subject matter.

Less frequently observed are emotions typically associated with optimism or emotional engagement, such as anticipation, trust, surprise, and joy, each appearing at substantially lower average counts. Notably, joy and surprise show the lowest average presence among all emotional categories.

These results imply that authors with sustained contributions to suicide research in 2004 predominantly expressed negative emotional tones, potentially reflecting the gravitas and emotionally charged nature of the topic. The findings underscore the affective burden embedded in academic discourse around suicide and may have implications for researcher mental health, narrative framing, and public perception of the field.

Distribution of Emotion Category Over 1990-2004:

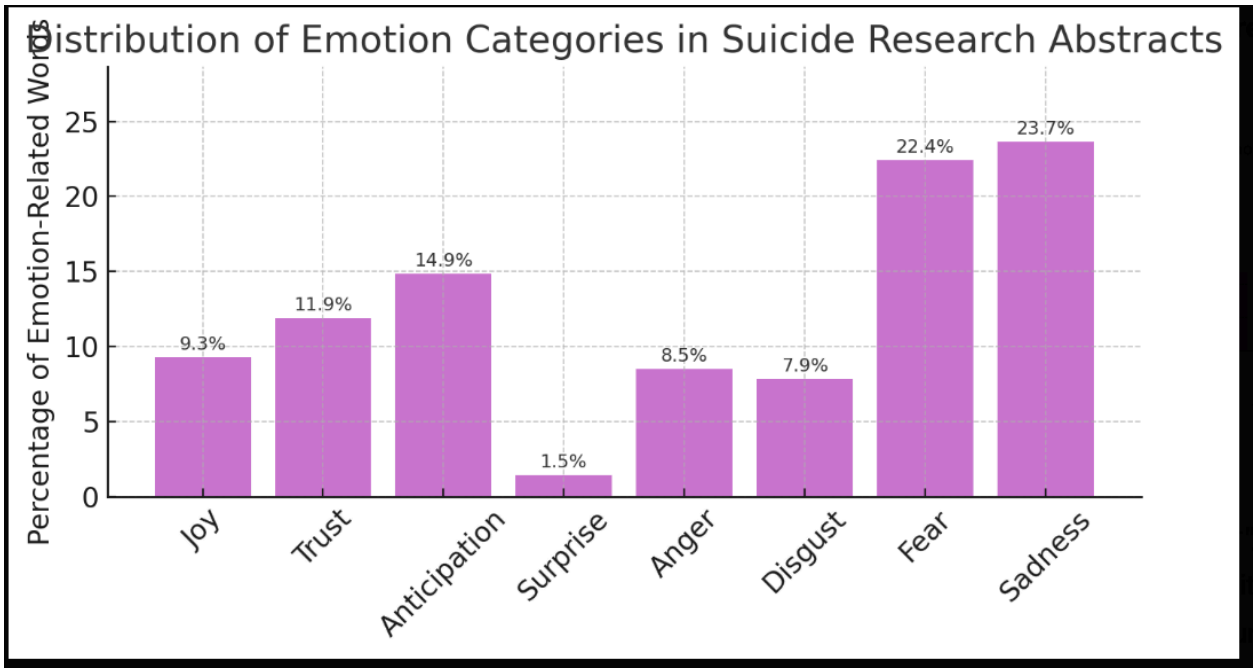

Figure G

This bar chart displays the proportion of emotion-related words present in suicide research abstracts published between 1990 and 2004, encompassing contributions from all authors during this period. Each emotion's value represents its relative frequency as a percentage of all emotion-related terms identified in the corpus.

The findings highlight a predominance of negative affective expressions within the abstracts. Sadness emerges as the most frequent emotion (23.7%), followed closely by fear (22.4%), together accounting for nearly half of all emotion-related words. These results underscore the emotionally charged nature of suicide-related discourse, reflecting the somber and distressing thematic content typical of this research domain.

Among the remaining negative emotions, anger (8.5%) and disgust (7.9%) appear at moderate levels, while surprise registers the lowest frequency (1.5%), indicating a minimal presence of unexpected or startling content in the emotional landscape of the texts.

On the positive end of the spectrum, anticipation (14.9%) is the most represented, suggesting some forward-looking or uncertain emotional framing in research narratives. Trust (11.9%) and joy (9.3%) appear to a lesser extent, though their combined total is substantially lower than the top negative emotions.

Overall, the distribution suggests a research field deeply embedded in negative emotional framing, consistent with the gravity of the topic. These results may inform future bibliometric and psychosocial investigations into how emotion-laden language in scientific literature shapes

scholarly communication, researcher well-being, and public engagement with mental health discourse.

<u>Emotion Comparison of Prolific vs. Non-Prolific Authors:</u>

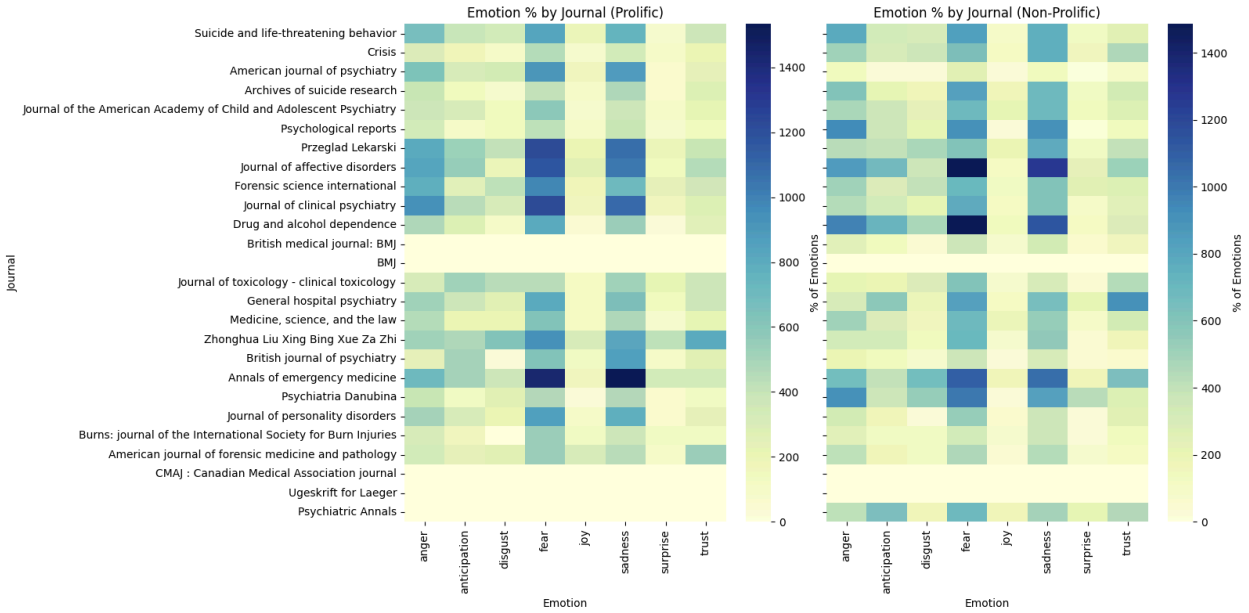

Figure H

Figure H presents a side-by-side heatmap comparison of emotional language used in suicide research articles published in the year 2004, disaggregated by author productivity. The left panel displays emotion distributions for articles authored by prolific researchers—those who published three or more papers in 2004—while the right panel shows corresponding data for non-prolific researchers (fewer than three publications). Each heatmap captures the percentage of emotion-related words per journal, categorized into eight basic emotions: anger, anticipation, disgust, fear, joy, sadness, surprise, and trust.

Overall, both groups exhibit dominant use of fear and sadness-related language across journals, consistent with the emotionally charged subject matter of suicide research. However, distinctions emerge in the concentration and dispersion of emotional tones.

Among prolific authors (left panel), journals such as Psychological Reports, Journal of Clinical Psychiatry, and Journal of Personality Disorders exhibit particularly high levels of fear and sadness. These authors also show notable but more isolated peaks in disgust and trust in select journals, such as the British Journal of Psychiatry and Journal of Toxicology - Clinical Toxicology.

In contrast, non-prolific authors (right panel) display more pronounced variability and stronger peaks across multiple emotions and journals. For instance, Archives of Suicide Research, Forensic Science International, and Journal of Clinical Psychiatry feature higher intensities of fear, sadness, and disgust, suggesting heightened emotional expressivity or less editorial standardization. Notably, the presence of anticipation and joy, though still limited, is more widely distributed in the non-prolific group.

These patterns suggest that prolific authors may adopt a more emotionally restrained or consistent academic tone, potentially shaped by experience, disciplinary norms, or editorial expectations. Non-prolific authors, on the other hand, may introduce more emotional variability, possibly reflecting diverse disciplinary backgrounds or less frequent engagement with suicide-related discourse.

The results highlight how publication frequency and journal venue influence emotional framing in suicide research, offering insight into authorship dynamics and the broader rhetorical landscape of mental health scholarship in 2004.

<u>Emotion Comparison of Prolific vs. Non-Prolific Authors Represented as a Percentage:</u>

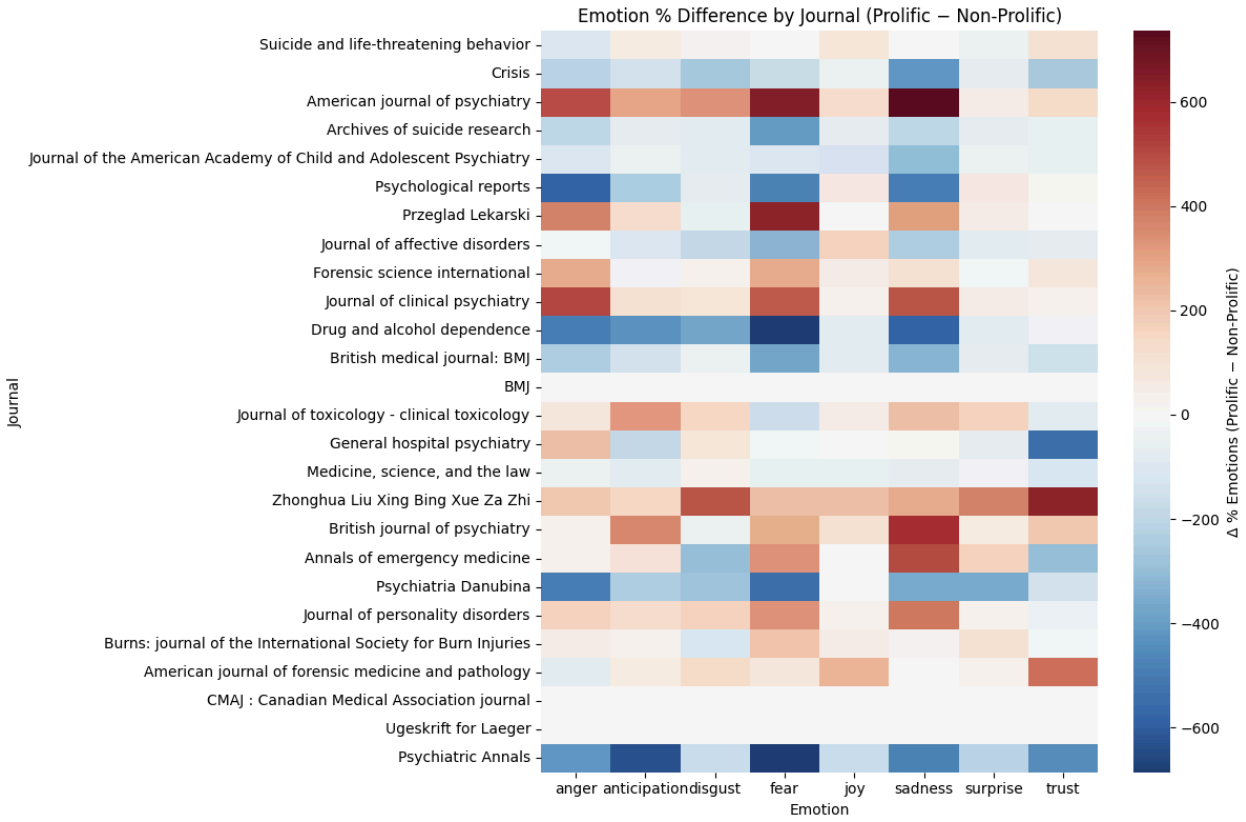
Emotion % Difference by Journal (Prolific − Non-Prolific)
Suicide and life-threatening behavior
Crisis
American journal of psychiatry
Archives of suicide research
Journal of the American Academy of Child and Adolescent Psychiatry
Psychological reports
Przeglad Lekarski
Journal of affective disorders
Forensic science international
Journal of clinical psychiatry
Drug and alcohol dependence
British medical journal: BMJ
BMJ
Journal of toxicology - clinical toxicology
General hospital psychiatry
Medicine, science, and the law
Zhonghua Liu Xing Bing Xue Za Zhi
British journal of psychiatry
Annals of emergency medicine
Psychiatria Danubina
Journal of personality disorders
Burns: journal of the International Society for Burn Injuries
American journal of forensic medicine and pathology
CMAJ : Canadian Medical Association journal
Ugeskrift for Laeger
Psychiatric Annals
Journal
anger
anticipation
disgust
fear
joy
sadness
surprise
trust
Emotion
600
400
200
0
−200
−400
−600
Δ % Emotions (Prolific − Non-Prolific)

Figure I

Figure I is a heatmap which illustrates the difference in emotional language usage between prolific and non-prolific authors across suicide research journals in the year 2004. Values represent the difference in emotion-related word counts as a percentage of total emotional terms per journal, computed as:
Δ% = (Prolific − Non-Prolific).
Positive values (shaded red) indicate a higher proportion of a given emotion in articles by prolific authors, while negative values (shaded blue) reflect greater usage by non-prolific authors.

The figure reveals several journal-specific divergences in emotional tone. Notably, prolific authors in journals such as Archives of Suicide Research and Journal of Personality Disorders expressed substantially more fear, disgust, and sadness compared to non-prolific counterparts, suggesting a deeper engagement with emotionally intense language. Conversely, non-prolific authors showed markedly higher levels of fear and sadness in journals like Psychological Reports, Journal of Clinical Psychiatry, and Psychiatric Annals, as evidenced by the strong blue gradients.

Journals such as American Journal of Psychiatry and British Journal of Psychiatry show substantial negative deltas in joy, sadness, and trust, suggesting more effective variation among non-prolific authors in those venues. On the other hand, prolific authors exhibited more consistent or elevated emotional language in journals like Drug and Alcohol Dependence, General Hospital Psychiatry, and Zhonghua Liu Xing Bing Xue Za Zhi, where warm tones (reds) dominate across several emotions.

These differential patterns suggest that author productivity in 2004 may influence both the intensity and type of emotional framing used across journals. The variation may stem from stylistic differences, familiarity with editorial expectations, or thematic specialization. Such disparities have implications for how emotional tone is communicated and perceived in the literature, shaping the rhetorical landscape of suicide-related scholarship within and across disciplines.

<u>Sentiment Polarity:</u>

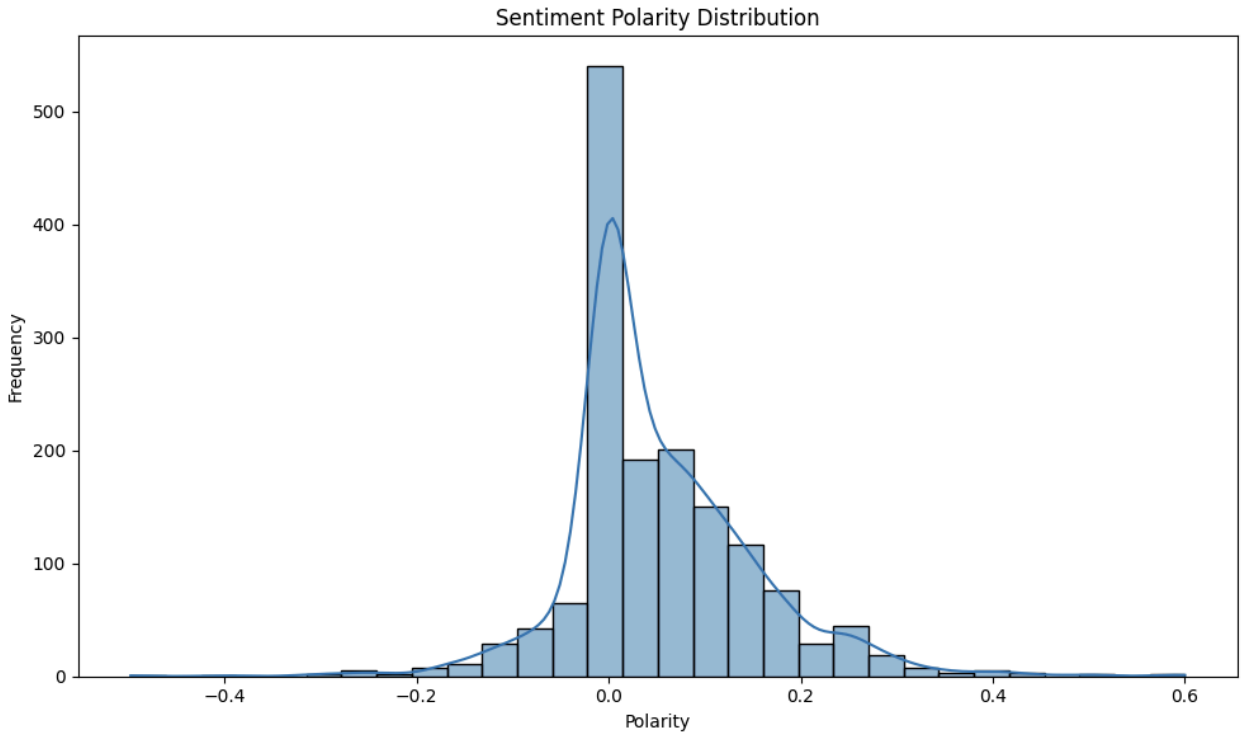

Figure J

This histogram in Figure J visualizes the distribution of sentiment polarity scores for suicide research abstracts published in the year 2004. Sentiment polarity is a continuous metric ranging from –1 (maximally negative) to +1 (maximally positive), with zero representing neutral sentiment. The histogram is overlaid with a kernel density estimate (KDE) curve to aid in identifying the underlying shape of the distribution.

The distribution is heavily right-skewed, with a pronounced central peak at or near zero, indicating that the majority of abstracts exhibit neutral sentiment. However, there is a clear asymmetry: the tail on the right (positive polarity) is longer and more populated than the tail on the left (negative polarity), suggesting that mildly positive sentiment is more common than strongly negative sentiment in 2004 suicide-related publications.

While extreme polarity values are rare on both ends, the higher frequency of slightly positive polarity values may reflect efforts by researchers to convey messages of prevention, intervention, or hope, even within the inherently sensitive and somber context of suicide research.

The overall narrow concentration around neutral sentiment and the lack of strongly negative outliers underscore a tendency toward objective or clinical tone, consistent with academic writing conventions. Nevertheless, the slight rightward tilt raises questions about optimistic framing strategies or positive bias in research dissemination, meriting further investigation into how sentiment relates to topics, authorship, and journal characteristics.

Top & Bottom 10 Sentiment Polarity by Journals Amongst Prolific Authors:

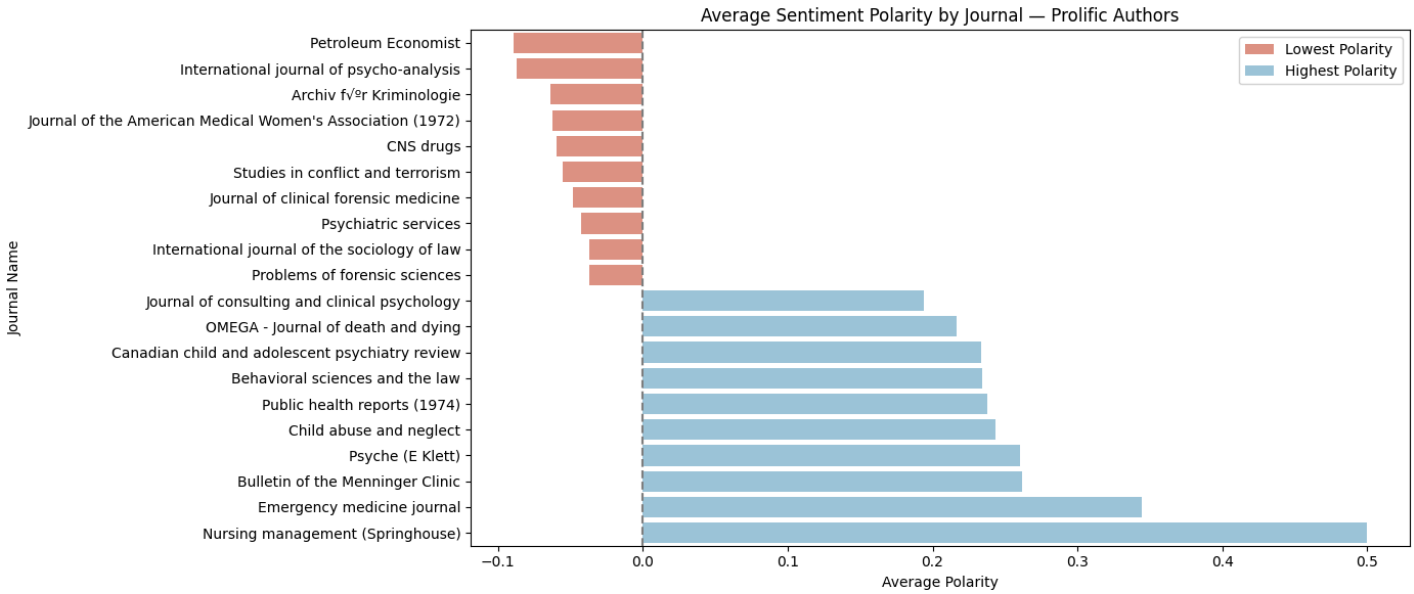

Figure K

Figure K presents the average sentiment polarity scores by journal, specifically among publications authored by prolific contributors in the dataset. Sentiment polarity values range from −1 (indicating negative sentiment) to +1 (indicating positive sentiment), with 0 representing neutral sentiment. This analysis provides insight into the emotional tone of scholarly writing across different journals within the context of suicide-related research.

Notably, Nursing Management exhibited the highest average sentiment polarity, with a mean score approaching +0.5, suggesting that articles published in this journal generally conveyed a more positive or optimistic tone. Similarly, journals such as Emergency Medicine Journal, Bulletin of the Menninger Clinic, and Psyche also displayed relatively high sentiment scores, indicating a trend toward positive framing or language usage within these outlets.

In contrast, journals such as Petroleum Economist, International Journal of Psycho-Analysis, and Archiv für Kriminologie Exhibited negative average sentiment polarity, with Petroleum Economist reaching the lowest mean score of approximately −0.1. These results may reflect a more critical, clinical, or problem-focused narrative style prevalent in these journals.

This divergence in sentiment polarity across journals may be attributed to differences in disciplinary focus, editorial tone, or thematic framing. For example, journals with clinical or therapeutic orientations may use more encouraging or empathetic language, while those focused on criminology, forensic science, or macroeconomic reporting may adopt a more neutral or negative tone due to the nature of their subject matter.

An important observation from Figure K is the asymmetry between the most positive and most negative journals in terms of average sentiment polarity. While the bottom 10 journals exhibit modest negative sentiment scores with the lowest value around −0.1, the top 10 journals show substantially higher positive polarity scores, several of which exceed +0.3, and with the most positive journal nearing +0.5.

This suggests that the magnitude of positive sentiment in the most optimistic journals is considerably stronger than the magnitude of negative sentiment in the most pessimistic ones. In other words, although some journals lean toward negativity, their sentiment remains only mildly negative, whereas positively skewed journals display more emphatic affective expression.

Such disparity may indicate an underlying bias in how sentiment is expressed in academic literature. Positive language, when present, tends to be more pronounced, possibly due to efforts to emphasize progress, hope, or therapeutic outcomes especially in health-related journals. In contrast, negative sentiment appears to be more constrained, perhaps due to academic norms around objectivity and neutral reporting, even when discussing distressing topics like suicide, mental illness, or violence.

This sentiment imbalance could also reflect disciplinary differences in tone and framing, where medical and public health journals tend to emphasize patient outcomes and intervention efficacy, while criminology or psychoanalytic journals may adopt a more analytical or detached tone that lacks strong emotional polarity.

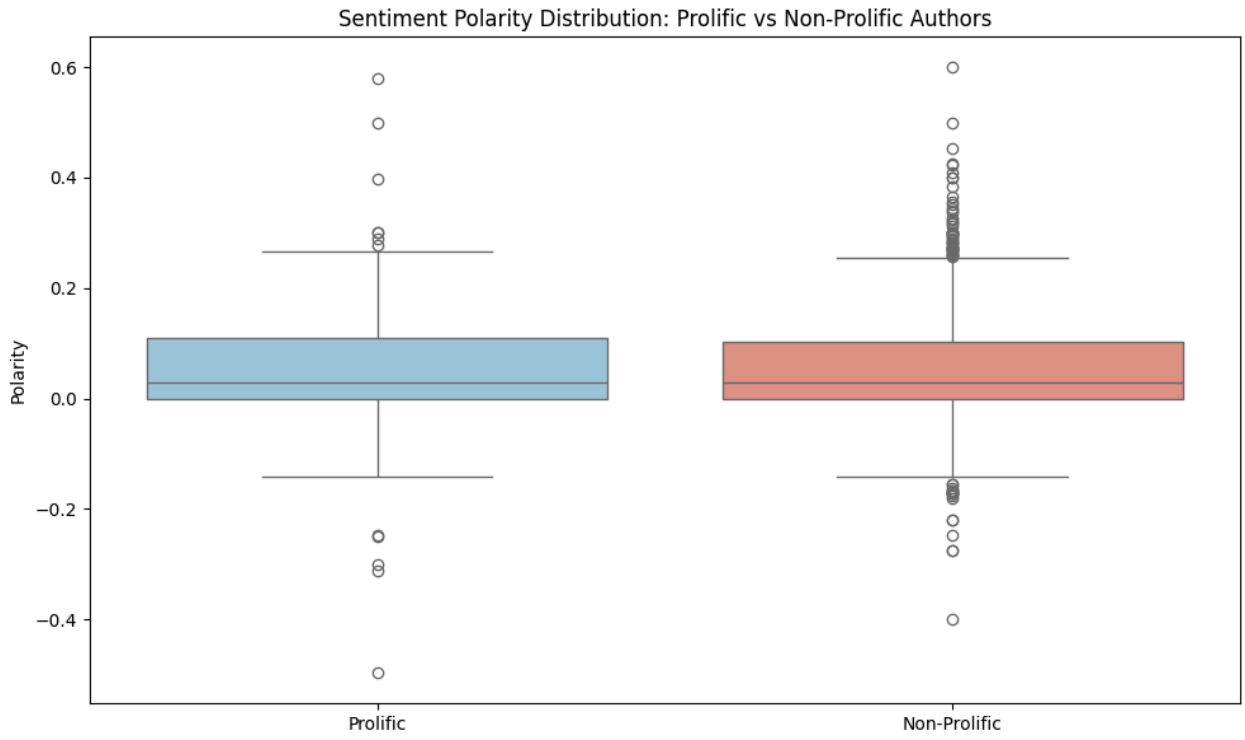

Figure L

Figure L compares the distribution of sentiment polarity scores in publications authored by prolific versus non-prolific researchers. Both author groups exhibit similarly centered distributions, with median polarity values slightly above zero, indicating a general tendency toward neutral-to-positive sentiment across all authors. However, a closer examination reveals subtle but noteworthy distinctions. The interquartile range (IQR) representing the middle 50% of values is slightly wider for non-prolific authors, suggesting greater variability in sentiment among this group. Prolific authors appear to have fewer extreme outliers, and their upper quartile reaches lower peak polarity values compared to non-prolific authors. This may suggest a more consistent and possibly more moderated emotional tone in their writing. Conversely, non-prolific authors display more pronounced high-end outliers, with polarity values exceeding +0.6. This suggests a subset of non-prolific contributors adopt more emotionally expressive language in select articles.

The overall distributional similarity implies no dramatic difference in average sentiment, but the increased variability and emotional range among non-prolific authors may reflect stylistic or thematic diversity potentially due to broader topical exploration or less editorial consistency. These findings highlight a potential distinction in writing tone and affective expression between established and occasional contributors, warranting further investigation into how author experience influences language use in scientific communication.

Average Sentiment Polarity & Subjectivity by Journal (Prolific Authors)

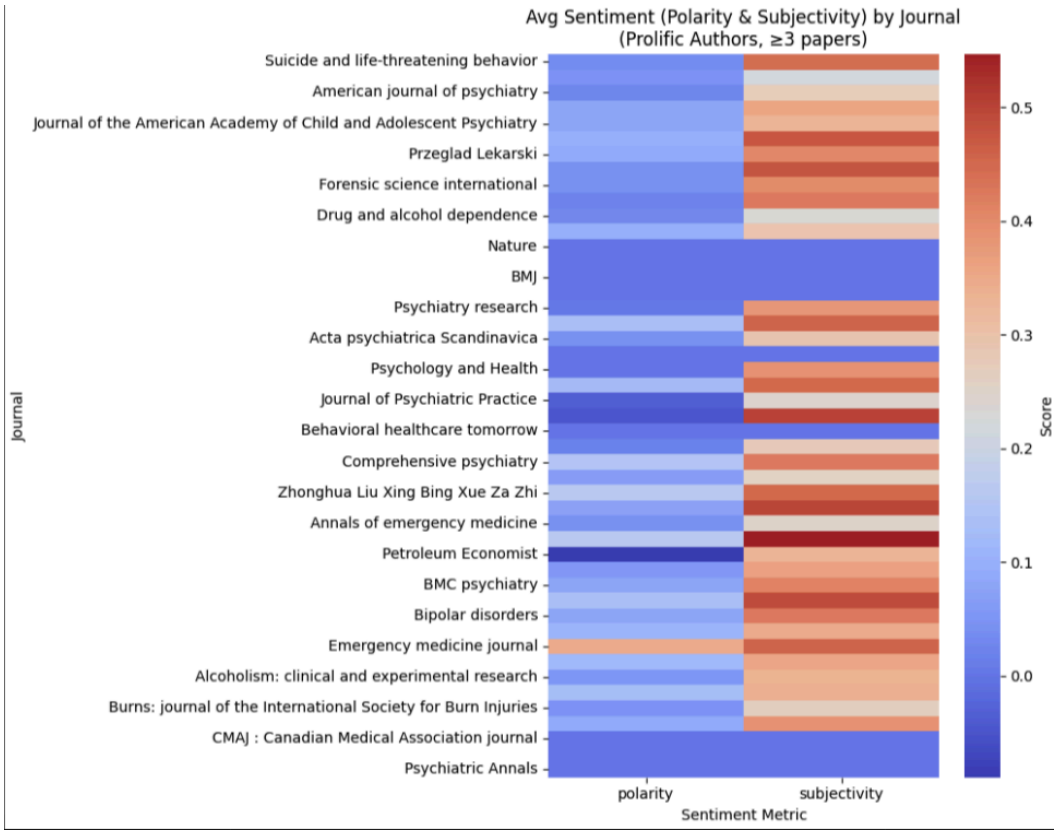

Figure M

Figure M presents a heatmap of average sentiment polarity and subjectivity scores across journals with publications from prolific authors. Subjectivity scores capture the degree of opinionated or subjective language, with higher values indicating less objectivity.

Most journals exhibit low average polarity values, with the majority of entries falling into the cool-blue range of the scale. This suggests that the overall tone across these journals is neutral to slightly negative, which may reflect the clinical and often somber nature of topics addressed, particularly in mental health and forensic disciplines.

Unlike polarity, subjectivity scores show more pronounced variation. Journals such as Zhonghua Liu Xing Bing Xue Za Zhi, Petroleum Economist, and BMJ demonstrate notably higher levels of subjectivity (warmer colors), suggesting that writing in these venues incorporates more interpretive, evaluative, or narrative language. In contrast, outlets like the Journal of Psychiatric Practice and Nature have much lower subjectivity scores, indicating more objective or fact-focused writing styles.

The data does not suggest a direct correlation between sentiment polarity and subjectivity. For example, Petroleum Economist shows low polarity (neutral to negative) but high subjectivity,

implying emotionally neutral yet opinion-laden writing. Conversely, Drug and Alcohol Dependence exhibits slightly more positive polarity yet moderate subjectivity.

Journals in clinical psychiatry and addiction (Comprehensive Psychiatry, Bipolar Disorders, Alcoholism: Clinical and Experimental Research) generally show low polarity and moderate-to-high subjectivity, potentially reflecting the complex, interpretive nature of clinical case analysis and treatment discussions.

This visualization underscores the nuanced language characteristics across academic journals in suicide and mental health research. While sentiment remains relatively flat or neutral, subjectivity varies more widely, suggesting that disciplinary norms and editorial policies may influence how interpretive or objective academic prose tends to be.

Network Analysis:

In order to better understand the interconnectivity of our data, we utilized PyVis to generate interactive network graphs. This visualization method allows us to explore how authors are connected to each other via co-authorship and how papers and grants that fund them relate to each other.

Coauthorship network:

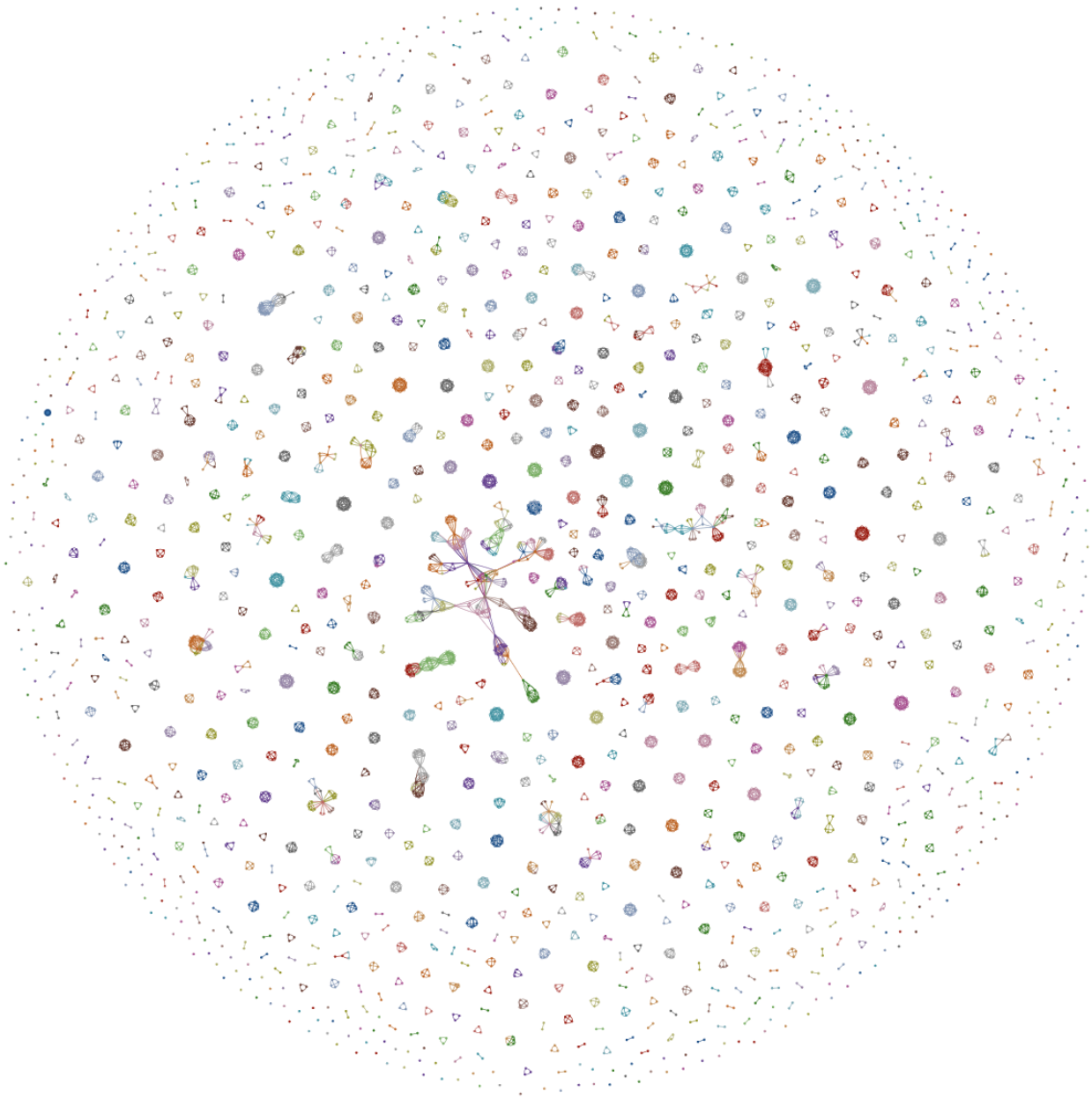

Figure N

Figure N depicts a network graph representing the co-authorship structure among researchers in the analyzed dataset. Each node corresponds to an individual author, and edges represent co-authorship links formed through shared publications. Nodes are grouped and color-coded based on communities or clusters identified via modularity-based community detection algorithms, reflecting cohesive subgroups of collaborating researchers.

The network exhibits a highly fragmented structure, with a large number of small, disconnected components and localized clusters, suggesting a field with many isolated or semi-isolated research groups. A central core of densely connected nodes is visible near the center of the graph, indicating the presence of highly collaborative author clusters likely anchored by prolific or institutionally central researchers. These core clusters often exhibit higher degree centrality and betweenness, serving as bridges between more sparsely connected regions.

The outer perimeter of the graph is populated by peripheral nodes and dyads, representing either authors with limited collaboration histories or single-article contributors. The diversity in cluster size and density highlights varying collaboration norms across research topics or institutions.

This network structure supports the interpretation that suicide-related academic research, as represented in this corpus, is distributed across multiple semi-independent research communities, with limited cross-group integration. Understanding the topology of this co-authorship network provides valuable insight into the collaborative dynamics, knowledge silos, and potential pathways for interdisciplinary engagement in the field.

Grant Network:

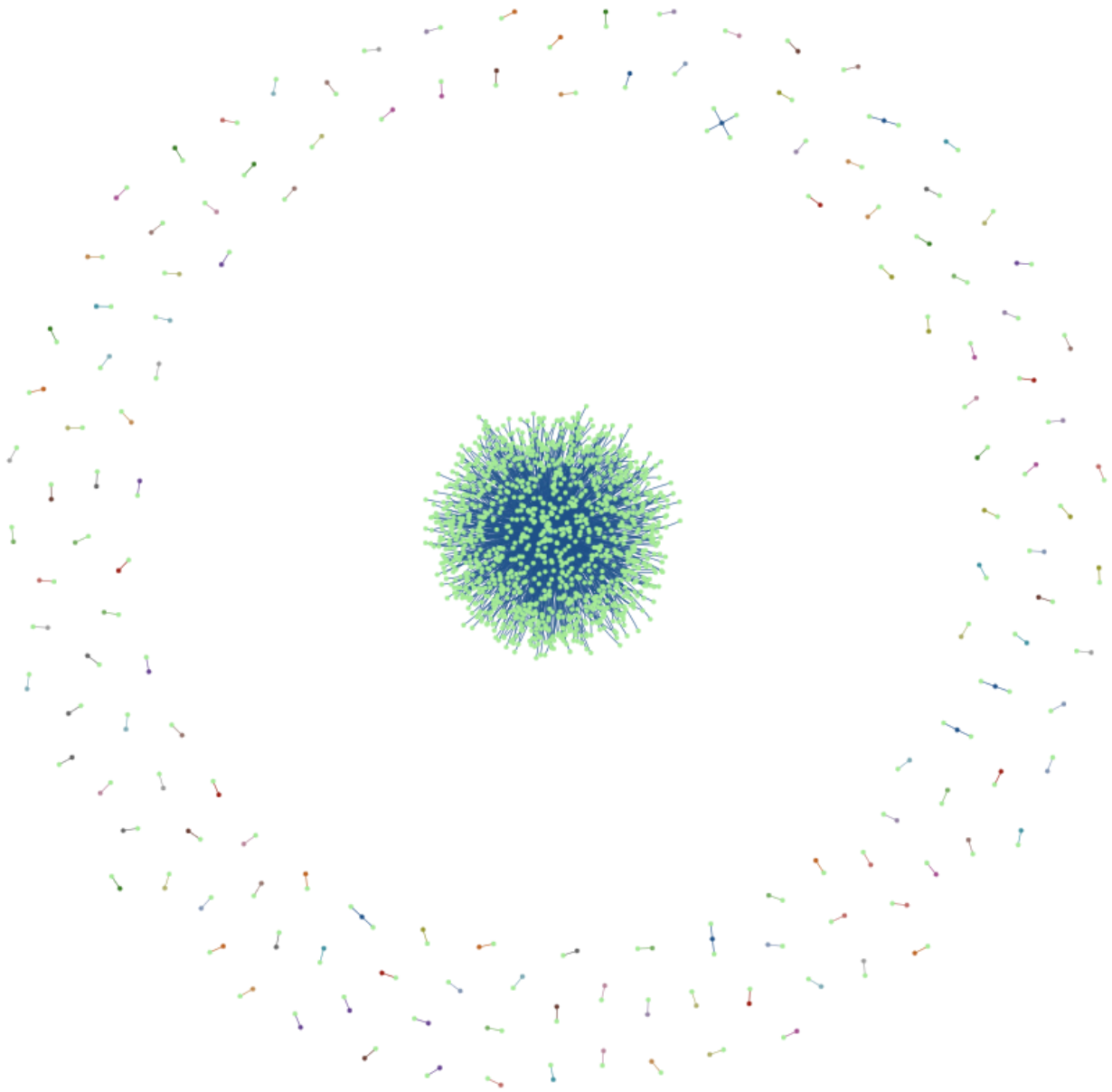

Figure O

Figure O represents the relationship between research papers and the grants that funded them in the year 2004. Each green node corresponds to an individual research paper, while each non-green node represents one or more grants associated with a given paper. Edges indicate direct funding relationships between papers and their respective grants.

At the center of the graph is a dense, tightly clustered core composed exclusively of papers for which no grant information was found. This central aggregation highlights the substantial proportion of suicide-related research published in 2004 that lacked explicit or extractable

funding acknowledgments, pointing to limitations in metadata completeness, inconsistent funding disclosures, or the prevalence of unfunded studies in the field.

Surrounding the core is a radial periphery made up of distinct clusters, each representing one or more grants linked to one or more funded papers. Most peripheral nodes (grants) are connected to a single paper, forming dyadic structures, though some appear to fund multiple publications, indicated by hub-like structures with multiple links. These outer-ring clusters reflect the fragmented and sparse nature of grant reporting in the dataset, with relatively few grants supporting multiple papers.

Overall, this visualization underscores the contrast between well-documented funding and a larger body of unfunded or unreported research activity. The network highlights opportunities for improved funding traceability and suggests that a considerable portion of suicide research in 2004 may have been conducted without formal grant support or with underreported grant affiliations.

Conclusion & Future Work

This project presents a thorough, data-rich exploration of suicide-themed academic literature from 1990 to 2004. Utilizing extensive web scraping, meticulous data cleaning, and sophisticated natural language processing tools, we crafted a refined dataset of over 16,000 articles. Our findings revealed significant publication rate trends, methodology changes, author dynamics, and the emotional tone of academic work.

The primary findings indicated a consistent increase in suicide studies over the decades, most notably emphasized by increasingly robust quantitative approaches and a surge in the number of publications in the early 2000s. Our sentiment analysis demonstrated that scholarly writing maintains an objective tone, revealing clear trends among journals and between prolific and non-prolific authors.

Despite the strength of our analysis, limitations arise from missing metadata, the absence of a DOI, and inconsistencies in abstract access. These limitations highlight the need for scholarly repositories to standardize the sharing of metadata.

Future projects will expand our dataset to include data beyond 2004, allowing for better analysis of recent trends, particularly considering increased mental health awareness and the impact of post-pandemic studies. Moreover, including citation metrics and enhanced emotion and topic modeling accuracy will provide further insight. Our design also serves as a solid foundation for

similar studies addressing various public health issues, thus rendering this project effective and versatile.

This research establishes a novel meta-research paradigm by integrating computational platforms with public health science, focusing on the evolving dynamics of suicide research. It offers authoritative guidance to researchers, practitioners, and policymakers working towards resolving one of the most pressing public health issues of our time.

Appendix

Appendix 1: SafetyLit Scraper

```
import os
import time
import re
import json
import csv
import requests
from bs4 import BeautifulSoup

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait, Select
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

################################################################################
# Utility Functions
################################################################################
```

```
def create_folder(folder="./safetylit_records"):
    if not os.path.exists(folder):
        os.makedirs(folder)
    return folder

def setup_driver():
    options = Options()
    # Uncomment the next line to run headless if desired:
    # options.add_argument("--headless")
    options.add_argument("user-agent=Mozilla/5.0 (X11; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/88.0.4324.96 Safari/537.36")
    service = Service(ChromeDriverManager().install())
    driver = webdriver.Chrome(service=service, options=options)
    return driver

def extract_ids(html):
    soup = BeautifulSoup(html, "html.parser")
    return [tag["title"] for tag in soup.find_all("abbr", class_="unapi-id") if
tag.get("title")]

def parse_bibtex_entries(bibtex_text):
    entries = []
    raw_entries = bibtex_text.split("@article")
    for raw in raw_entries:
        raw = raw.strip()
        if not raw:
            continue
        entry_text = "@article" + raw
        entry_data = {}
        lines = entry_text.splitlines()
        for line in lines:
            if "=" in line:
                try:
                    key, value = line.split("=", 1)
                    key = key.strip().lower()
                    value = value.strip().lstrip("{").rstrip("},").strip('"')
                    entry_data[key] = value
                except Exception:
                    continue
        entries.append(entry_data)
    return entries

def separate_authors(author_str):
    if not author_str:
        return None
    return [a.strip() for a in author_str.split(" and ") if a.strip()]
```

```
def parse_records_text(text):
    match = re.search(r"Records\s+(\d+)-(\d+)\s+of\s+(\d+)", text)
    if match:
        start = int(match.group(1))
        end = int(match.group(2))
        total = int(match.group(3))
        return (start, end, total)
    return (0, 0, 0)

def get_total_pages(driver):
    try:
        td_element = driver.find_element(By.XPATH,
'//*[@id="paginationForm"]/table/tbody/tr[1]/td')
        records_text = td_element.text
        print("Pagination text:", records_text)
        start_num, end_num, total_records = parse_records_text(records_text)
        display_count = 10
        if total_records:
            computed_pages = (total_records // display_count) + (1 if total_records
% display_count else 0)
            return computed_pages
        else:
            return 10
    except Exception as e:
        print("Error in get_total_pages:", e)
        return 10

def get_session_and_pages(driver):
    session = None
    for cookie in driver.get_cookies():
        if cookie["name"] == "PHPSESSID":
            session = cookie["value"]
            break
    pages = get_total_pages(driver)
    return session, pages

def find_next_button(driver, current_page):
    buttons = driver.find_elements(
        By.XPATH,
        '//*[@id="paginationForm"]//input[@type="button" and @class="submit" and
@value=">"]'
    )
    if not buttons:
        raise Exception("No next buttons found at all.")
    for btn in buttons:
        onclick = btn.get_attribute("onclick")  # e.g., doSubmit('3')
```

```
        m = re.search(r"doSubmit\('(\d+)'\)", onclick)
        if m:
            target_page = int(m.group(1))
            if target_page > current_page:
                return btn
    raise Exception("No valid next button referencing page > current_page.")

################################################################################
# Main Scraper
################################################################################

def main():
    folder = create_folder("./safetylit_records")
    driver = setup_driver()

    # 1) Open the first search results page with all parameters.
    results_url =
"https://www.safetylit.org/citations/index.php?fuseaction=citations.summaries_perfo
rmance&thesfilter=1&find_1=suicide&find_2=self-harm&find_3=suicidal&field_1=textwor
d&find_13=&find_14=&find_15=&op_1=not&find_4=Aptosis&find_5=Artistic&find_6=Assiste
d&field_2=textword&find_16=Business&find_17=Cellular&find_18=Economic&op_2=not&find
_7=Educational&find_8=Financial&find_9=Fiscal&field_3=textword&find_19=Literary&fin
d_20=Molecular&find_21=National&op_3=not&find_10=Political&find_11=Professional&fin
d_12=Social&field_4=textword&find_22=Substrate&find_23=&find_24=&from=1990&to=2004&
pub_type=none&categories%5B%5D=24&categories%5B%5D=26&categories%5B%5D=23&categorie
s%5B%5D=25&categories%5B%5D=1&categories%5B%5D=27&categories%5B%5D=2&categories%5B%
5D=3&categories%5B%5D=4&categories%5B%5D=5&categories%5B%5D=36&categories%5B%5D=28&
categories%5B%5D=35&categories%5B%5D=34&categories%5B%5D=29&categories%5B%5D=6&cate
gories%5B%5D=30&categories%5B%5D=39&categories%5B%5D=7&categories%5B%5D=37&categori
es%5B%5D=31&categories%5B%5D=38&categories%5B%5D=32&categories%5B%5D=8&categories%5
B%5D=40&categories%5B%5D=9&categories%5B%5D=10&categories%5B%5D=33&categories%5B%5D
=22&categories%5B%5D=11&categories%5B%5D=12&categories%5B%5D=13&categories%5B%5D=14
&categories%5B%5D=15&categories%5B%5D=16&categories%5B%5D=17&categories%5B%5D=21&ca
tegories%5B%5D=18&categories%5B%5D=41&categories%5B%5D=19&categories%5B%5D=20"
    driver.get(results_url)
    time.sleep(3)
    print("DEBUG: Current URL:", driver.current_url)
    driver.get(results_url)
    time.sleep(3)
    print("DEBUG: current_url =", driver.current_url)

    # 2) Parse pagination text to determine total pages
    try:
        td_element = driver.find_element(By.XPATH,
'//*[@id="paginationForm"]/table/tbody/tr[1]/td')
        records_text = td_element.text
        print("Pagination text:", records_text)
```

```
        start_num, end_num, total_records = parse_records_text(records_text)
        if total_records == 0:
            print("Could not parse total records from the page. Defaulting to 10
pages.")
            total_pages = 10
        else:
            display_count = 10
            total_pages = (total_records // display_count) + (1 if total_records %
display_count else 0)
    except Exception as e:
        print("Error parsing pagination info:", e)
        total_pages = 10

    print(f"Determined total_pages (computed) = {total_pages}")
    # Force total_pages to 1617
    total_pages = 1617
    print(f"Overriding total_pages to: {total_pages}")

    all_records = []
    current_page = 1

    # 3) Loop through pages to extract data
    while current_page <= total_pages:
        html = driver.page_source
        record_ids = extract_ids(html)
        print(f"Page {current_page}: Found {len(record_ids)} record IDs.")

        if record_ids:
            post_data = {"export": "bib", "export_recs": "|".join(record_ids)}
            headers = {"User-Agent": "Mozilla/5.0"}
            bib_response = requests.post(
                "https://www.safetylit.org/unapi/conversion_all.php",
                data=post_data,
                headers=headers
            )
            if "@article" in bib_response.text:
                entries = parse_bibtex_entries(bib_response.text)
                for entry in entries:
                    authors = separate_authors(entry.get("author", None))
                    record_data = {
                        "title": entry.get("title", ""),
                        "author_details": "; ".join(authors) if authors else "",
                        "author_affiliations": entry.get("affiliation", ""),
                        "journal_name": entry.get("journal", ""),
                        "publication_year": entry.get("year", ""),
                        "language": entry.get("language", ""),
                        "keywords": entry.get("keywords", ""),
```

```
                        "doi": entry.get("doi", ""),
                        "research_methodology": entry.get("methodology", ""),
                        "data_source": "SafetyLit",
                        "grant_info": entry.get("grant", ""),
                        "suicide_prevention_measures": entry.get("prevention", ""),
                        "abstract": entry.get("abstract", "")

                    }
                    all_records.append(record_data)
                print(f"Page {current_page}: Processed {len(record_ids)} record
IDs.")
            else:
                print(f"Page {current_page}: No valid BibTeX data returned.")
        else:
            print(f"Page {current_page}: No record IDs found on this page.")

        current_page += 1
        try:
            next_button = find_next_button(driver, current_page - 1)
            next_button.click()
            time.sleep(3)
        except Exception as e:
            print(f"No more next button or cannot click next button (page
{current_page-1}):", e)
            break

    # 4) Save all records to a master CSV file
    master_csv = os.path.join(folder, "raw_scrape_april_14.csv")
    fieldnames = ["title", "author_details", "author_affiliations", "journal_name",
"publication_year",
                  "language", "keywords", "doi", "research_methodology",
"data_source",
                  "grant_info", "suicide_prevention_measures", "abstract"]
    try:
        with open(master_csv, "w", newline="", encoding="utf-8") as csvfile:
            writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
            writer.writeheader()
            for record in all_records:
                writer.writerow(record)
        print(f"Master CSV saved: {master_csv}")
    except Exception as e:
        print("Error saving CSV:", e)

    # 5) Retrieve PHP session ID from cookies and total pages, then quit.
    session, pages = get_session_and_pages(driver)
    print(f"Session: {session}")
    print(f"Pages (from get_total_pages): {pages}")
```

```
    driver.quit()
    return session, pages

if __name__ == "__main__":
    import csv
    session, pages = main()
    print("Final session and pages:", session, pages)
```

Appendix 2 Main Code:

```
# -*- coding: utf-8 -*-
"""
Created on Tue Apr 15 07:53:39 2025

@author: W. Hall

IE 7475 Project Main File

This is the master code file that
runs all of our code in sequence,
making reproducing our group's
work simple and efficient.

This code will:
      1) Scrape SafetyLit for all available
      required data

      2) Use NLP and other sites to produce
      missing data points

      3) Clean and validate all data

"""

import os
import SafetyLit_Scraper, kwscrape, methscrape, doi_scraper
import grantscraper, preventionscraper, datavalidationfixed
from rich import print


def main(dir_path):
      print('Running primary scraper...')
```

```
        SafetyLit_Scraper.main()
        print('Running keyword scraper...')
        kwscrape.main(dir_path)
        print('Running methodology scraper...')
        methscrape.main(dir_path)
        print('Running DOI scraper...')
        doi_scraper.main(dir_path)
        print('Running grant scraper...')
        grantscraper.main(dir_path)
        print('Running prevention scraper...')
        preventionscraper.main(dir_path)
        print('Running data validation...')
        datavalidationfixed.main(dir_path)


 if __name__ == '__main__':
        # Set current directory as working directory
        dir_path = os.path.dirname(os.path.realpath(__file__)) +
 r'\safetylit_records'
        main(dir_path)
```

Appendix 3 NLP Modelling**:**

```
# -*- coding: utf-8 -*-
"""SingleYearNLPFINAL.ipynb

Automatically generated by Colab.

Original file is located at
   https://colab.research.google.com/drive/1bPPC2NTt6RRvWx-Q62fu3eaRm50mEIij
"""

# Install and download models in Colab
!pip install -q spacy textblob wordcloud openpyxl nrclex pyvis sentence-transformers
!python -m textblob.download_corpora
!python -m spacy download en_core_web_sm

import pandas as pd, spacy

# Load spaCy model
```

```
nlp = spacy.load('en_core_web_sm')

# 1) Read the new CSV
raw_df = pd.read_csv('/content/v4.2004_complete_data.csv')

# 2) Drop any completely empty columns
df = raw_df.dropna(axis=1, how='all')

# 3) Standardize all column names to lowercase with underscores,
#    so downstream code (e.g. df['abstract'], df['publication_year']) still works.
df.columns = (
   df.columns
     .str.strip()
     .str.lower()
     .str.replace(' ', '_')
     .str.replace('-', '_')
)

# 4) Split 'combined_author_details' into individual author_details.* columns
df['combined_author_details'] = df['combined_author_details'].astype(str)
authors_split = df['combined_author_details'].str.split(';', expand=True)
authors_split.columns = [f'author_details.{i+1}' for i in
range(authors_split.shape[1])]
df = pd.concat([df, authors_split], axis=1)

# inspect data
df.head()

# — Cell 2: Trim author_details & fix types —

# 1) Drop any author_details beyond the first 10 co-authors
author_cols_to_drop = [
   col for col in df.columns
   if col.startswith('author_details.') and int(col.split('.')[-1]) > 10
]
df.drop(columns=author_cols_to_drop, inplace=True)

# 2) Ensure abstracts are strings (replace missing with empty string)
df['abstract'] = df['abstract'].fillna('').astype(str)

# 3) Cast publication_year to numeric (so plotting by year works)
df['publication_year'] = (
```

```
   pd.to_numeric(df['publication_year'], errors='coerce')
     .astype('Int64')
)

# Preview
df.head()

# — Cell 3: Missing-values & Abstract-length exploration —

import seaborn as sns
import matplotlib.pyplot as plt
import plotly.express as px

# 1) Missing-values heatmap
plt.figure(figsize=(12, 6))
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Across Columns')
plt.xlabel('Columns')
plt.ylabel('Row Index')
plt.show()

# 2) Abstract-length distribution
df['abstract_length'] = df['abstract'].str.len()

fig = px.box(
   df,
   y='abstract_length',
   points='all',
   title='Abstract Length Distribution'
)
fig.show()

# 3) Outlier detection (1.5×IQR rule)
q1 = df['abstract_length'].quantile(0.25)
q3 = df['abstract_length'].quantile(0.75)
iqr = q3 - q1
upper_bound = q3 + 1.5 * iqr

outlier_count = df['abstract_length'].gt(upper_bound).sum()
print(f"Abstract-length outliers (# docs > {upper_bound:.0f} chars): {outlier_count}")

# — Cell 4: From WordCloud through Topic Modeling & Clustering —
```

```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import LatentDirichletAllocation, PCA
from sklearn.cluster import KMeans
import plotly.express as px

# 1) Word Cloud of all abstracts
all_text = ' '.join(df['abstract'].dropna())
wc = WordCloud(width=800, height=400, background_color='white').generate(all_text)
plt.figure(figsize=(15, 7))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Abstract Texts')
plt.show()

# 2) TF-IDF vectorization
tfidf = TfidfVectorizer(max_features=1000, stop_words='english')
X = tfidf.fit_transform(df['abstract'])

# 3) LDA topic modeling
lda = LatentDirichletAllocation(n_components=5, random_state=42)
lda.fit(X)

for idx, comp in enumerate(lda.components_, start=1):
   terms = tfidf.get_feature_names_out()
   top_terms = [terms[i] for i in comp.argsort()[-10:]]
   print(f"Topic {idx}: {top_terms}")

# 4) K-Means clustering
kmeans = KMeans(n_clusters=5, random_state=42)
df['cluster'] = kmeans.fit_predict(X)

# 5) PCA for 2D visualization
pca = PCA(n_components=2, random_state=42)
coords = pca.fit_transform(X.toarray())
df['pca1'], df['pca2'] = coords[:, 0], coords[:, 1]

fig = px.scatter(
   df,
   x='pca1',
```

```
   y='pca2',
   color=df['cluster'].astype(str),
   title='Abstract Clusters (KMeans + PCA)'
)
fig.show()

# — Cell 5: NER Label Frequency —

from collections import Counter
import seaborn as sns
import matplotlib.pyplot as plt

# Take a reproducible sample of abstracts
sample_text = ' '.join(
   df['abstract']
     .dropna()
     .sample(50, random_state=42)
)

# Run spaCy NER
doc = nlp(sample_text)
labels = [ent.label_ for ent in doc.ents]

# Count label occurrences
freq = Counter(labels)

# Plot
plt.figure(figsize=(10, 6))
sns.barplot(x=list(freq.keys()), y=list(freq.values()))
plt.title('NER Label Frequency (50-abstract sample)')
plt.xlabel('Entity Type')
plt.ylabel('Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# — Cell 6: Sentiment Polarity & Publication Trends —

from textblob import TextBlob
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# 1) Compute sentiment scores
df['polarity'] = df['abstract'].apply(lambda x: TextBlob(x).sentiment.polarity)
df['subjectivity'] = df['abstract'].apply(lambda x:
TextBlob(x).sentiment.subjectivity)

# 2) Plot polarity distribution
plt.figure(figsize=(10, 6))
sns.histplot(df['polarity'], bins=30, kde=True)
plt.title('Sentiment Polarity Distribution')
plt.xlabel('Polarity')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# 3) Plot number of publications by year
yearly_counts = df['publication_year'].value_counts().sort_index()

plt.figure(figsize=(10, 6))
plt.plot(yearly_counts.index, yearly_counts.values, marker='o')
plt.title('Publications by Year')
plt.xlabel('Year')
plt.ylabel('Number of Publications')
plt.grid(True)
plt.tight_layout()
plt.show()

# — Cell 7: Co-authorship Network & Prolific Authors —

import networkx as nx
import matplotlib.pyplot as plt
from collections import Counter
import pandas as pd

# 1) Identify all author_details columns
author_cols = [col for col in df.columns if col.startswith('author_details.')]

# 2) Build the co-authorship graph
G = nx.Graph()
for _, row in df.iterrows():
   authors = [row[col] for col in author_cols if pd.notna(row[col]) and row[col] !=
'']
   for i in range(len(authors)):
```

```
        for j in range(i+1, len(authors)):
            G.add_edge(authors[i], authors[j])

# 3) Plot the network
plt.figure(figsize=(12, 12))
pos = nx.spring_layout(G, k=0.5, seed=42)
nx.draw(G, pos, node_size=30, edge_color='gray', with_labels=False)
plt.title('Co-authorship Network Graph')
plt.axis('off')
plt.show()

# 4) Compute prolific authors (more than one publication)
all_authors = df[author_cols].values.ravel()
all_authors = pd.Series(all_authors).dropna().astype(str)
author_counts = Counter(all_authors)
prolific = {author: count for author, count in author_counts.items() if count > 1}

# 5) Create a DataFrame and display top 20
prolific_df = (
   pd.DataFrame.from_dict(prolific, orient='index', columns=['publications'])
     .sort_values('publications', ascending=False)
     .rename_axis('author')
     .reset_index()
)
print("Top Prolific Authors:")
display(prolific_df.head(20))

# — Cell 8: Emotion Detection per Prolific Author —

from nrclex import NRCLex
import pandas as pd
import matplotlib.pyplot as plt
from collections import Counter

# 1) Rebuild an author-level table with combined abstracts
records = []
author_cols = [c for c in df.columns if c.startswith('author_details.')]

for author, count in prolific.items():
   # find all rows where this author appears
   mask = df[author_cols].eq(author).any(axis=1)
   abstracts = df.loc[mask, 'abstract'].dropna().astype(str)
```

```
    combined = ' '.join(abstracts)
    records.append({
        'author': author,
        'publications': count,
        'text': combined
    })

author_df = pd.DataFrame(records)

# 2) Emotion extraction function
def detect_emotions(text):
    try:
        return NRCLex(text).raw_emotion_scores
    except:
        return {}

# 3) Apply to each author's combined text
author_df['emotion_scores'] = author_df['text'].apply(detect_emotions)

# 4) Expand into separate columns
emotions = pd.DataFrame(author_df['emotion_scores'].tolist()).fillna(0).astype(int)
author_df = pd.concat([author_df.drop(columns='emotion_scores'), emotions], axis=1)

# 5) Show top-5 authors by 'joy'
print("Top 5 Prolific Authors by Joy:")
display(author_df.sort_values('joy', ascending=False)[
    ['author', 'publications', 'joy']
].head(5))

# 6) Plot average emotion across all prolific authors
avg_emotions = emotions.mean().sort_values(ascending=False)

plt.figure(figsize=(8, 5))
avg_emotions.plot(kind='bar')
plt.title('Average Emotion Scores (Prolific Authors)')
plt.xlabel('Emotion')
plt.ylabel('Average Count')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# — Cell 9: Journal Publication Breakdown: Prolific vs Non-Prolific —
```

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# 1) Identify rows with any prolific author
author_cols = [col for col in df.columns if col.startswith('author_details.')]
mask_prolific_rows = df[author_cols].isin(prolific.keys()).any(axis=1)
prolific_rows_df = df.loc[mask_prolific_rows].copy()
non_prolific_df   = df.loc[~mask_prolific_rows].copy()

# 2) Count documents per journal
prolific_counts     =
prolific_rows_df['journal_name'].value_counts().rename('prolific_count')
non_prolific_counts =
non_prolific_df['journal_name'].value_counts().rename('non_prolific_count')

# 3) Merge and compute totals and percentages
journal_merged = pd.concat([prolific_counts, non_prolific_counts], axis=1).fillna(0)
journal_merged['total']             = journal_merged['prolific_count'] +
journal_merged['non_prolific_count']
journal_merged['percent_prolific']  = (journal_merged['prolific_count'] /
journal_merged['total']) * 100

# 4) Filter journals with at least 5 papers
filtered = journal_merged[journal_merged['total'] >= 5]

# 5) Plot top 20 journals by % prolific authors
plt.figure(figsize=(12, 6))
sns.barplot(
   x=filtered['percent_prolific'].head(20),
   y=filtered.index[:20],
   palette='Blues_d'
)
plt.title('% of Publications by Prolific Authors per Journal (min 5 papers)')
plt.xlabel('% Prolific Authors')
plt.ylabel('Journal')
plt.tight_layout()
plt.show()

# — Cell 10: Sentiment Polarity by Journal (Prolific Authors) —
```

```
import seaborn as sns
import matplotlib.pyplot as plt

# 1) Compute average polarity by journal for prolific authors
sentiment_by_journal = prolific_rows_df.groupby('journal_name')['polarity'].mean()

# 2) Sort and select bottom and top 10 journals
sentiment_sorted = sentiment_by_journal.sort_values()
bottom10 = sentiment_sorted.head(10)
top10    = sentiment_sorted.tail(10)

# 3) Plot
plt.figure(figsize=(14, 6))
sns.barplot(x=bottom10.values, y=bottom10.index, color='salmon', label='Lowest
Polarity')
sns.barplot(x=top10.values,    y=top10.index,    color='skyblue', label='Highest
Polarity')
plt.axvline(0, color='gray', linestyle='--')
plt.title('Average Sentiment Polarity by Journal — Prolific Authors')
plt.xlabel('Average Polarity')
plt.ylabel('Journal Name')
plt.legend()
plt.tight_layout()
plt.show()

# — Cell 11: Sentiment Polarity Comparison: Prolific vs Non-Prolific —

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from textblob import TextBlob
import numpy as np

# 1) Recompute the two groups if needed
author_cols      = [c for c in df.columns if c.startswith('author_details.')]
mask_prolific    = df[author_cols].isin(prolific.keys()).any(axis=1)
prolific_df      = df.loc[mask_prolific].copy()
non_prolific_df  = df.loc[~mask_prolific].copy()

# 2) Compute sentiment polarity
prolific_df['polarity']     = prolific_df['abstract'].apply(lambda x:
TextBlob(str(x)).sentiment.polarity)
```

```
non_prolific_df['polarity'] = non_prolific_df['abstract'].apply(lambda x:
TextBlob(str(x)).sentiment.polarity)

# 3) Build a long-form DataFrame
comp_df = pd.DataFrame({
   'polarity': np.concatenate([prolific_df['polarity'], non_prolific_df['polarity']]),
   'group':    ['Prolific']*len(prolific_df) + ['Non-Prolific']*len(non_prolific_df)
})

# 4) Plot
plt.figure(figsize=(10, 6))
sns.boxplot(
   x='group',
   y='polarity',
   data=comp_df,
   palette={'Prolific':'skyblue','Non-Prolific':'salmon'}
)
plt.title('Sentiment Polarity Distribution: Prolific vs Non-Prolific Authors')
plt.xlabel('')
plt.ylabel('Polarity')
plt.tight_layout()
plt.show()

# — Cell 12: Average Sentiment Heatmap by Journal —

import seaborn as sns
import matplotlib.pyplot as plt

# 1) Compute average polarity & subjectivity per journal (prolific authors)
sentiment_by_journal = (
   prolific_df
   .groupby('journal_name')[['polarity', 'subjectivity']]
   .mean()
)

# 2) Keep journals with at least 3 papers
counts = prolific_df['journal_name'].value_counts()
eligible = counts[counts >= 3].index
sentiment_by_journal = sentiment_by_journal.loc[eligible]

# 3) Plot heatmap
plt.figure(figsize=(10, 8))
```

```
sns.heatmap(
   sentiment_by_journal,
   annot=False,
   fmt=".2f",
   cmap="coolwarm",
   cbar_kws={'label': 'Score'}
)
plt.title('Avg Sentiment (Polarity & Subjectivity) by Journal\n(Prolific Authors, ≥3
papers)')
plt.xlabel('Sentiment Metric')
plt.ylabel('Journal')
plt.tight_layout()
plt.show()

# — Cell 13: Emotion Proportions by Journal —

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from nrclex import NRCLex

# Define the eight NRC emotion columns
emotion_cols = [
   'anger', 'anticipation', 'disgust', 'fear',
   'joy', 'sadness', 'surprise', 'trust'
]

# 0) Compute row-level emotion scores (if not already present)
#    This builds a small DF of shape (n_rows × 8)
row_emotions = (
   df['abstract']
     .fillna('')
     .astype(str)
     .apply(lambda text: pd.Series(NRCLex(text).raw_emotion_scores))
     .fillna(0)
     .astype(int)
)

# 1) Concatenate these back onto df
df_em = pd.concat([df, row_emotions[emotion_cols]], axis=1)

# 2) Re-split into prolific vs non-prolific rows
```

```
author_cols      = [c for c in df_em.columns if c.startswith('author_details.')]
mask_prolific    = df_em[author_cols].isin(prolific.keys()).any(axis=1)
prol_em_df       = df_em.loc[mask_prolific]
non_prol_em_df   = df_em.loc[~mask_prolific]

# 3) Sum emotions per journal
emo_pro = prol_em_df.groupby('journal_name')[emotion_cols].sum()
emo_non = non_prol_em_df.groupby('journal_name')[emotion_cols].sum()

# 4) Keep only journals with ≥3 papers in both groups
cnt_pro = prol_em_df['journal_name'].value_counts()
cnt_non = non_prol_em_df['journal_name'].value_counts()
common  = cnt_pro[cnt_pro >= 3].index.intersection(cnt_non[cnt_non >= 3].index)

emo_pro = emo_pro.loc[common]
emo_non = emo_non.loc[common]

# 5) Convert sums to percentages
prop_pro = emo_pro.div(cnt_pro, axis=0).loc[common] * 100
prop_non = emo_non.div(cnt_non, axis=0).loc[common] * 100

# 6) Plot side-by-side heatmaps
fig, axes = plt.subplots(1, 2, figsize=(16, 8), sharey=True)

sns.heatmap(
   prop_pro, ax=axes[0],
   cmap='YlGnBu', annot=False, fmt=".1f",
   cbar_kws={'label':'% of Emotions'}
)
axes[0].set_title('Emotion % by Journal (Prolific)')
axes[0].set_xlabel('Emotion')
axes[0].set_ylabel('Journal')

sns.heatmap(
   prop_non, ax=axes[1],
   cmap='YlGnBu', annot=False, fmt=".1f",
   cbar_kws={'label':'% of Emotions'}
)
axes[1].set_title('Emotion % by Journal (Non-Prolific)')
axes[1].set_xlabel('Emotion')
axes[1].set_ylabel('')
```

```
plt.tight_layout()
plt.show()

# — Cell 14: Emotion Difference Heatmap —

import seaborn as sns
import matplotlib.pyplot as plt

# 1) Compute difference (Prolific % - Non-Prolific %) — prop_pro and prop_non must
exist
emotion_diff = prop_pro.subtract(prop_non, fill_value=0)

# 2) Plot heatmap of differences
plt.figure(figsize=(12, 8))
sns.heatmap(
   emotion_diff,
   cmap='RdBu_r',
   center=0,
   annot=False,
   fmt=".1f",
   cbar_kws={'label': 'Δ % Emotions (Prolific - Non-Prolific)'}
)
plt.title('Emotion % Difference by Journal (Prolific - Non-Prolific)')
plt.xlabel('Emotion')
plt.ylabel('Journal')
plt.tight_layout()
plt.show()

# — Cell 15: Grant-Paper Bipartite Network (PyVis) —

from pyvis.network import Network
import pandas as pd
import matplotlib.cm as cm
import matplotlib.colors as mcolors

# 1) Build a DataFrame of grant-paper edges
df['grant_information'] = df['grant_information'].fillna('').astype(str)
edges = []
for idx, grants in df['grant_information'].items():      # ← use .items() here
   for grant in grants.split(';'):
       grant = grant.strip()
       if grant and grant.lower() not in ('none', 'nan', ''):
```

```
            edges.append({'grant': grant, 'paper_id': f"Paper_{idx}"})

grant_edges_df = pd.DataFrame(edges)

# 2) Count papers per grant
grant_counts = grant_edges_df['grant'].value_counts()

# 3) Create color map & size scaling
cmap = cm.get_cmap('tab20', len(grant_counts))
grant_color_map = {
    grant: mcolors.rgb2hex(cmap(i))
    for i, grant in enumerate(grant_counts.index)
}
max_count = grant_counts.max()
grant_size_scaled = {
    grant: 10 + (count / max_count) * 40
    for grant, count in grant_counts.items()
}

# 4) Initialize PyVis network
net = Network(
    height="750px", width="100%",
    bgcolor="#ffffff", font_color="black",
    notebook=True
)

# 5) Add grant nodes
for grant, count in grant_counts.items():
    net.add_node(
        n_id=grant, label=grant,
        color=grant_color_map[grant],
        size=grant_size_scaled[grant],
        title=f"{grant} — {count} papers"
    )

# 6) Add paper nodes
for pid in grant_edges_df['paper_id'].unique():
    net.add_node(n_id=pid, label=pid, color='lightgreen', size=10)

# 7) Add edges
for _, row in grant_edges_df.iterrows():
    net.add_edge(row['grant'], row['paper_id'])
```

```
# 8) Display
net.show_buttons(filter_=['physics'])
net.show("grant_network.html")

# — Cell 16: Interactive Co-Authorship Network via PyVis —

from pyvis.network import Network
from collections import Counter
import matplotlib.cm as cm
import matplotlib.colors as mcolors
import pandas as pd

# 1) Identify author columns and build edge list
author_cols = [c for c in df.columns if c.startswith('author_details.')]
edges = []
for _, row in df.iterrows():
   authors = [row[c] for c in author_cols if pd.notna(row[c]) and row[c] != '']
   for i in range(len(authors)):
       for j in range(i+1, len(authors)):
           edges.append((authors[i], authors[j]))

# 2) Compute publication counts per author
all_authors = pd.Series(df[author_cols].values.ravel()).dropna().astype(str)
author_counts = Counter(all_authors)

# 3) Prepare node sizes (10-50 px) and colors (categorical)
max_ct = max(author_counts.values())
size_map = {a: 10 + (cnt/max_ct)*40 for a,cnt in author_counts.items()}
cmap = cm.get_cmap('tab20', len(author_counts))
color_map = {a: mcolors.rgb2hex(cmap(i)) for i,a in enumerate(author_counts)}

# 4) Initialize PyVis network
net = Network(
   height="750px",
   width="100%",
   bgcolor="#ffffff",
   font_color="black",
   notebook=True
)

# 5) Add author nodes
```

```
for author, cnt in author_counts.items():
    net.add_node(
        n_id=author,
        label=author,
        size=size_map[author],
        color=color_map[author],
        title=f"{cnt} publications"
    )

# 6) Add co-author edges
for a, b in edges:
    net.add_edge(a, b)

# 7) Show controls and render
net.show_buttons(filter_=['physics'])
net.show("coauthorship_network.html")
```

Appendix 4 Keyword Scraper:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.firefox.options import Options
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.select import Select
from time import sleep
import pandas as pd
import os


def getKeywords(author, title, driver):
      pubmed = 'https://pubmed.ncbi.nlm.nih.gov/advanced/'

      print(f'Starting search for: {str(title)}')
      sleep(0.5)

      print('\tLoading page...')
      authone = str(author).split(' and ')[0]
      title = str(title)
      driver.get(pubmed)

      sleep(1)
```

```
      print('\tExecuting search...')
      select_el = driver.find_element(By.ID, 'field-selector')
      select = Select(select_el)

      select.select_by_value('Author')
      searchbar = driver.find_element(By.ID, 'id_term')
      searchbar.send_keys(authone + Keys.ENTER)

      select.select_by_value('Title')
      searchbar.send_keys(title + Keys.ENTER)

      button = driver.find_element(By.CLASS_NAME, 'search-btn')
      button.click()

      keys = []
      print('\tLooking for keywords...')
      try:
      mesh = driver.find_element(By.ID,

'mesh-terms').find_element(By.CLASS_NAME,
                                                                'keywords-list')

      buttons = mesh.find_elements(By.TAG_NAME, 'button')

      for button in buttons:
            keys.append(button.text.strip())
      sleep(1)
      except Exception:
      keys.append('N\\A')
      sleep(1)
      print(f'Keywords found: {keys}')

      return keys


def main(file_path):

      options = Options()
      options.set_preference("browser.helperApps.neverAsk.saveToDisk",
"text/bib")
      options.set_preference("browser.helperApps.alwaysAsk.force", False)

      driver = webdriver.Firefox(options=options)
```

```
	df = pd.read_csv(file_path + r'\all_records.csv')
	for index, row in df.iterrows():
	if pd.isna(row['keywords']) or row['keywords'] == '':
		df.at[index, 'keywords'] = getKeywords(row['author_details'],
row['title'], driver)

		print('Writing to CSV...\n')
		df.to_csv(file_path + r'\bibwithkw.csv', index=False)

	else:
		print('Already found! Skipping...\n')

	print(df[['author_details', 'title', 'keywords']])
	df.to_csv(file_path + r'\bibwithkw.csv', index=False)

	driver.close()

if __name__ == '__main__':
	# Setup for web driver:
	options = Options()
	options.set_preference("browser.helperApps.neverAsk.saveToDisk",
"text/bib")
	options.set_preference("browser.helperApps.alwaysAsk.force", False)

	driver = webdriver.Firefox(options=options)

	main()
```

Appendix 5 Methodology Imputation:

```
import pandas as pd
import nltk
from nltk.corpus import wordnet as wn
import os

# Function to preprocess text and search for terms
def analyze_methodology(frame):
	# Define lists of terms associated with quantitative and qualitative
research
	quantitative_terms = [
	"statistical analysis", "regression analysis", "correlation",
```

```
"hypothesis testing",
      "survey", "questionnaire", "experiment", "control group", "randomized
controlled trial",
      "data points", "sample size", "standard deviation", "variance",
"p-value",
      "quantitative data", "numerical data", "metrics", "measurement",
"likert scale",
      "descriptive statistics", "inferential statistics", "t-test",
"anova", "chi-square test",
      "factor analysis", "multivariate analysis", "cross-sectional study",
"longitudinal study",
      "big data", "data mining", "epidemiological study", "risk factors",
"suicide rates",
      "self-harm rates", "psychometric scales", "predictive modeling",
"effect size",
      "standardized instruments", "meta-analysis", "systematic review"
      ]

      qualitative_terms = [
      "interviews", "focus groups", "case study", "ethnography",
"phenomenology",
      "grounded theory", "narrative analysis", "content analysis",
"thematic analysis",
      "discourse analysis", "observation", "field notes", "participant
observation",
      "open-ended questions", "qualitative data", "textual analysis",
"interpretive research",
      "contextual analysis", "inductive reasoning", "coding",
"triangulation",
      "thick description", "emic perspective", "etic perspective",
"narrative analysis",
      "thematic analysis", "grounded theory", "phenomenology",
"interpretive phenomenological analysis",
      "contextual factors", "stigma", "coping mechanisms", "help-seeking
behavior"
      ]

      # Update list with synonyms:

      for item in quantitative_terms:
      try:
            syns = wn.synset(f'{item}.n.01').lemma_names()
            for i in syns:
```

```
                    if i not in quantitative_terms:
                    quantitative_terms.append(i)
        except Exception:
              pass

        for item in qualitative_terms:
        try:
              syns = wn.synset(f'{item}.n.01').lemma_names()
              for i in syns:
                    if i not in qualitative_terms:
                    qualitative_terms.append(i)
        except Exception:
              pass

        text = str(frame.abstract)  # Convert to lowercase for
case-insensitive matching
        # Initialize counters for quantitative and qualitative terms
        quant_count = 0
        qual_count = 0

        # Check for quantitative terms
        for term in quantitative_terms:
        if term in text:
              quant_count += 1

        # Check for qualitative terms
        for term in qualitative_terms:
        if term in text:
              qual_count += 1

        # Determine the methodology based on term counts
        if quant_count > qual_count:
        return "Quantitative"
        elif qual_count > quant_count:
        return "Qualitative"
        else:
        return "Mixed Methods or Unclear"


def main(dir_path):
        df = pd.read_csv(dir_path + r'\all_records.csv')

        meth = df.apply(analyze_methodology, axis=1)
```

```
      df['methodology'] = meth

      df.to_csv(dir_path + r'\bibwithmeth.csv')


if __name__ == '__main__':
      dir_path = os.path.dirname(os.path.realpath(__file__)) +
r'\safetylit_records'

      main(dir_path)
```

Appendix 6 DOI Scraper:

```
import os
import pandas as pd
import requests
import time
import urllib.parse
from requests.exceptions import RequestException

# Function to clean titles for API queries
def clean_title(title):
      if isinstance(title, str):
      title = title.replace("&", "and")  # Replace ampersand
      title = title.replace("'", "")  # Remove single quotes
      title = title.replace('"', "")  # Remove double quotes
      title = title.replace("?", "")  # Remove question marks
      title = urllib.parse.quote(title)  # URL encode the title
      return title

# Function to fetch DOI from CrossRef API with retries
def get_doi_from_crossref(issn, title, max_retries=3):
      base_url = "https://api.crossref.org/works"
      query = f"?query.title={clean_title(title)}&filter=issn:{issn}"
      headers = {"User-Agent": "DOI-Finder/1.0
(mailto:your-email@example.com)"}

      for attempt in range(max_retries):
      try:
            response = requests.get(base_url + query, headers=headers,
timeout=30)  # Increased timeout
```

```
            response.raise_for_status()
            data = response.json()

            if "message" in data and "items" in data["message"] and
len(data["message"]["items"]) > 0:
                  return data["message"]["items"][0].get("DOI", None)

      except RequestException as e:
            print(f"Error fetching DOI for ISSN {issn} and title {title}
(Attempt {attempt+1}): {e}")
            time.sleep(5)  # Wait before retrying

      return None  # Return None if all attempts fail


def main(file_path):
      # Process missing DOIs
      df = pd.read_csv(file_path+ r'\v3.scrubbed_data_with_grant_info.csv')

      for index, row in df.iterrows():
      if pd.isna(row["doi"]) and pd.notna(row["issn"]) and
pd.notna(row.get("title")):
            doi = get_doi_from_crossref(row["issn"], row["title"])
            if doi:
                  df.at[index, "doi"] = doi
            time.sleep(1)  # To avoid hitting API rate limits
      # Save dataset
      updated_doi_file_path = dir_path + "bibwithdoi.csv"
      df.to_csv(updated_doi_file_path, index=False)

      print(f"DOI retrieval complete! Updated dataset saved as
{updated_doi_file_path}")

if __name__ == '__main__':
      # Load the dataset
      dir_path = os.path.dirname(os.path.realpath(__file__)) +
r'\safetylit_records'

      main(dir_path)
```

Appendix 7 Grant Scraper:

```
import pandas as pd
import os
import requests
from bs4 import BeautifulSoup
import time
from IPython.display import display

def search_pubmed(title, retries=3, delay=5):
      """Searches PubMed for an article by title and extracts grant/funding
information, with retry logic."""
      search_url = f'https://pubmed.ncbi.nlm.nih.gov/?term={title.replace("
", "+")}'
      print(f"--- DEBUG: Results Page URL --- {search_url}")
      headers = {'User-Agent': 'Mozilla/5.0'}

      for attempt in range(retries):
      try:
            response = requests.get(search_url, headers=headers,
timeout=10)
            response.raise_for_status()
            break
      except requests.exceptions.RequestException as e:
            print(f"Attempt {attempt+1} failed: {e}")
            time.sleep(delay)
      else:
      return None, "none found"

      soup = BeautifulSoup(response.text, 'html.parser')

      # Get first article link
      article_link = soup.find('a', {'class': 'docsum-title'})
      if not article_link:
      return None, "none found"

      article_url = 'https://pubmed.ncbi.nlm.nih.gov' +
article_link.get('href', '')
      print(f"--- DEBUG: Article Page URL --- {article_url}")

      for attempt in range(retries):
      try:
            article_response = requests.get(article_url, headers=headers,
timeout=10)
            article_response.raise_for_status()
```

```
            break
      except requests.exceptions.RequestException as e:
            print(f"Attempt {attempt+1} failed: {e}")
            time.sleep(delay)
      else:
      return article_url, "none found"

      article_soup = BeautifulSoup(article_response.text, 'html.parser')

      funding_section = article_soup.find('div', {'id': 'grants'})
      if not funding_section:
      print("--- DEBUG: Searching for alternative funding sections ---")
      funding_section = article_soup.find('section', string=lambda text:
text and 'Funding' in text)
      if not funding_section:
      funding_section = article_soup.find('div', class_='funding-info')

      if not funding_section:
      print("--- DEBUG: Grants section not found! ---")
      funding_info = "none found"
      else:
      print("--- DEBUG: Grants section found! Extracting data... ---")
      funding_info = '\n'.join([
            f"Grant: {a.get('data-ga-label', a.text.strip())} | Details:
{a.get('title', 'No details available')} | Text: {a.text.strip()} (Link:
https://pubmed.ncbi.nlm.nih.gov{a.get('href', '')})"
            for a in funding_section.find_all(['a', 'div', 'p'])
      ])

      return article_url, funding_info

def main(file_path):
      df = pd.read_csv(file_path + r'\all_records.csv')

      # Iterate through the titles in CSV
      df['PubMed_URL'] = "none found"
      df['ISSN'] = df.get('ISSN', 'none found')
      df['ISBN'] = df.get('ISBN', 'none found')
      df['Funding_Info'] = "none found"

      for i, title in enumerate(df['title']):  # Adjust index for correct
row placement
      if pd.isna(title):
```

```
            continue

        url, funding = search_pubmed(title)
        df.at[i, 'PubMed_URL'] = url
        df.at[i, 'Funding_Info'] = funding

        print(f"Processed {i+1}/{len(df)}: {title}")
        time.sleep(2)  # Respectful delay to avoid being blocked

        # Display results as a DataFrame
        display(df)
        print("Scraping completed. Results displayed as DataFrame.")

        # Count the number of rows where 'Funding_Info' is not "none found"
        grant_count = (df['Funding_Info'] != "none found").sum()

        print(f"Total number of grant sections found: {grant_count}")

        # Define the output file path
        output_path = dir_path + r'\bibwithgrant.csv'

        # Save the DataFrame to a CSV file
        df.to_csv(output_path, index=False)

        print(f"Data saved successfully to {output_path}")

if __name__ == '__main__':
        # Define file path
        dir_path = os.path.dirname(os.path.realpath(__file__)) +
r'\safetylit_records'


        # Load CSV into a DataFrame and get articles 8001 to 12000

        main()
```

Appendix 8 Prevention Method Imputation:

```
import matplotlib.pyplot as plt
import os
import pandas as pd
```

```
import spacy
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from collections import Counter
from langdetect import detect
from deep_translator import GoogleTranslator
from sklearn.feature_extraction.text import TfidfVectorizer


def check_removed_abstracts(df, unwanted_texts):
      remaining_unwanted =
df[df['Abstract'].str.lower().isin(unwanted_texts) |
df['Abstract'].str.strip().eq("") | df['Abstract'].isna()]
      if remaining_unwanted.empty:
      print("All unwanted and blank abstracts have been successfully
removed.")
      else:
      print("Some unwanted abstracts remain in the dataset:")
      print(remaining_unwanted[['Title', 'Abstract']])


# Function to detect language and translate if necessary
def translate_to_english(text):
      try:
      detected_lang = detect(text)  # Detect language
      if detected_lang != "en":  # Only translate non-English abstracts
            translated_text = GoogleTranslator(source='auto',
target='en').translate(text)
            if translated_text and translated_text.lower() != text.lower():
# Ensure translation changed the text
                  return translated_text, True  # Return translated text
and flag
      return text, False  # Return original text and flag
      except Exception as e:
      return text, False  # If detection or translation fails, return
original


# Function to count and display translated abstracts
def count_translated_abstracts(df):
      translated_df = df[df['was_translated']]
      print(f"Number of abstracts translated: {len(translated_df)}")
      if not translated_df.empty:
```

```
    print("Titles of translated abstracts:")
    print(translated_df[['Title', 'Abstract',
'translated_abstract']].head(10))  # Show first 10 for review


# Function to download translated abstracts for manual review
def download_translated_abstracts_csv(df):
    translated_df = df[['Title', 'Abstract', 'translated_abstract',
'was_translated']]
    translated_df.to_csv(dir_path +
r'\translated_abstracts_for_review.csv', index=False)
    print("Translated abstracts saved as
'translated_abstracts_for_review.csv'")


# Preprocessing function
def preprocess_text(text):
    if pd.isna(text):
    return ""
    text = text.lower()  # Convert to lowercase
    text = re.sub(r'[^a-zA-Z0-9\s]', '', text)  # Remove special
characters
    return text

# Apply preprocessing


# Tokenization and stopword removal
def tokenize_text(text):
    nlp = spacy.load("en_core_web_sm")
    doc = nlp(text)
    tokens = [token.lemma_ for token in doc if not token.is_stop and
token.is_alpha]  # Remove stopwords & keep words only
    return " ".join(tokens)

# Apply tokenization


# Function to download tokenized abstracts
def download_tokenized_csv(df):
    tokenized_df = df[['Title', 'Abstract', 'translated_abstract',
'cleaned_abstract', 'tokenized_abstract']]
    tokenized_df.to_csv(dir_path + r'\tokenized_abstracts.csv',
```

```
index=False)
      print("Tokenized abstracts saved as 'tokenized_abstracts.csv'")


# Comprehensive List of Suicide Prevention and Mental Health Treatment
Keywords



# Function to extract suicide prevention methods using keyword matching
def extract_prevention_methods(text, prevention_keywords):
      found_methods = [word for word in prevention_keywords if word in
text.lower()]
      return ", ".join(set(found_methods)) if found_methods else None  #
Remove duplicates


# Function to extract prevention-related terms using regex
def extract_prevention_methods2(text, therapy_pattern,
crisis_services_pattern, medication_pattern, holistic_pattern,
medical_terms_pattern):
      therapy_matches = re.findall(therapy_pattern, text,
flags=re.IGNORECASE)
      crisis_matches = re.findall(crisis_services_pattern, text,
flags=re.IGNORECASE)
      medication_matches = re.findall(medication_pattern, text,
flags=re.IGNORECASE)
      holistic_matches = re.findall(holistic_pattern, text,
flags=re.IGNORECASE)
      medical_terms_matches = re.findall(medical_terms_pattern, text,
flags=re.IGNORECASE)

      all_matches = list(set(therapy_matches + crisis_matches +
medication_matches + holistic_matches + medical_terms_matches))  # Remove
duplicates
      return ", ".join(all_matches) if all_matches else None


def main(file_path):
      # 1 Therapy and Treatment Methods
      therapy_pattern = r"\b(?:cognitive behavioral therapy|cbt|dialectical
behavior therapy|dbt|psychodynamic therapy|interpersonal therapy|family
therapy|group therapy|supportive counseling|motivational
```

```
interviewing|problem-solving therapy|mindfulness-based cognitive
therapy|talk
therapy|psychotherapy|counseling|therapy|analysis|healing|remedy|therapeuti
cs|mental therapy|personal therapy|spiritual therapy|psychosomatic
therapy|telepathic therapy|psychic therapy|mental treatment|counseling
psychology|mental health therapy|psychiatric counseling|psychiatric
therapy|psychological therapy|psychiatric care|psychological support|mental
health services|depression treatment|self-harm prevention|substance abuse
treatment)\b"

      # 2 Crisis Intervention Services
      crisis_services_pattern = r"\b(?:suicide prevention hotline|crisis
intervention teams|emergency psychiatric services|mobile crisis units|peer
support programs|safety planning|crisis stabilization units|24/7 crisis
text lines|community mental health centers|assertive community
treatment|act teams)\b"

      # 3 Pharmaceuticals & Medications
      medication_pattern =
r"\b(?:fluoxetine|prozac|sarafem|sertraline|zoloft|citalopram|celexa|escita
lopram|lexapro|paroxetine|paxil|seroxat|fluvoxamine|luvox|venlafaxine|effex
or|duloxetine|cymbalta|desvenlafaxine|pristiq|levomilnacipran|fetzima|amitr
iptyline|elavil|endep|nortriptyline|pamelor|desipramine|norpramin|imipramin
e|tofranil|clomipramine|anafranil|doxepin|sinequan|trimipramine|surmontil|p
rotriptyline|vivactil|phenelzine|nardil|tranylcypromine|parnate|isocarboxaz
id|marplan|selegiline|emsam|bupropion|wellbutrin|zyban|mirtazapine|remeron|
nefazodone|serzone|trazodone|desyrel|oleptro|vilazodone|viibryd|vortioxetin
e|trintellix|haloperidol|haldol|chlorpromazine|thorazine|fluphenazine|proli
xin|perphenazine|trilafon|thioridazine|mellaril|thiothixene|navane|loxapine
|loxitane|pimozide|orap|trifluoperazine|stelazine|risperidone|risperdal|ola
nzapine|zyprexa|quetiapine|seroquel|aripiprazole|abilify|ziprasidone|geodon
|clozapine|clozaril|paliperidone|invega|asenapine|saphris|lurasidone|latuda
|iloperidone|fanapt|brexpiprazole|rexulti|cariprazine|vraylar|lithium|litho
bid|lamotrigine|lamictal|valproate|depakote|divalproex
sodium|carbamazepine|tegretol|oxcarbazepine|trileptal|gabapentin|neurontin|
topiramate|topamax|alprazolam|xanax|lorazepam|ativan|diazepam|valium|clonaz
epam|klonopin|chlordiazepoxide|librium|clorazepate|tranxene|oxazepam|serax|
temazepam|restoril|triazolam|halcion|buspirone|buspar|hydroxyzine|atarax|vi
staril|pregabalin|lyrica|meprobamate|miltown|equanil|propranolol|inderal|pr
azosin|minipress|methylphenidate|ritalin|concerta|daytrana|metadate|aptensi
o|dexmethylphenidate|focalin|amphetamine|adderall|mydayis|evekeo|dextroamph
etamine|dexedrine|vyvanse|lisdexamfetamine|desoxyn|methamphetamine|zolpidem
|ambien|eszopiclone|lunesta|zaleplon|sonata|ramelteon|rozerem|suvorexant|be
```

```
lsomra|trazodone|desyrel|doxepin|silenor|atomoxetine|strattera|guanfacine|i
ntuniv|clonidine|kapvay|modafinil|provigil|armodafinil|nuvigil)\b"

      # 4 Holistic & Alternative Therapies
      holistic_pattern = r"\b(?:acupuncture|meditation|mindfulness|yoga|tai
chi|qigong|herbal medicine|chinese
medicine|ayurveda|homeopathy|naturopathy|aromatherapy|massage
therapy|reiki|energy healing|hypnotherapy|chiropractic care|sound
therapy|music therapy|art therapy|animal-assisted therapy|equine
therapy|nature therapy|forest bathing|holistic nutrition|functional
medicine|integrative medicine|breathwork|progressive muscle
relaxation|guided imagery|biofeedback|light therapy|sensory deprivation
therapy|float therapy|spiritual counseling|faith-based
therapy|psychoneuroimmunology|vagus nerve stimulation|ayahuasca
therapy|psilocybin-assisted therapy|mdma-assisted therapy|ketamine
therapy|microdosing|craniosacral therapy|somatic
therapy|rolfing|feldenkrais method|alexander technique)\b"

      # 5 Medical Terminology Related to Mental Health
      medical_terms_pattern = r"\b(?:electroconvulsive
therapy|ect|repetitive transcranial magnetic stimulation|rtms|ketamine
therapy|psychiatric hospitalization|involuntary commitment|partial
hospitalization program|intensive outpatient program|iop|behavioral
activation|exposure therapy|systematic desensitization|schema
therapy|attachment-based therapy)\b"

      prevention_keywords = [
      # Psychotherapies and Counseling Approaches
      "cognitive behavioral therapy", "cbt", "dialectical behavior
therapy", "dbt",
      "psychodynamic therapy", "interpersonal therapy", "family therapy",
      "group therapy", "supportive counseling", "motivational
interviewing",
      "problem-solving therapy", "mindfulness-based cognitive therapy",
      "talk therapy", "psychotherapy", "counseling", "therapy", "analysis",
      "healing", "remedy", "therapeutics", "mental therapy", "personal
therapy",
      "spiritual therapy", "psychosomatic therapy", "telepathic therapy",
      "psychic therapy", "mental treatment", "counseling psychology",
      "mental health therapy", "psychiatric counseling", "psychiatric
therapy",
      "psychological therapy", "psychiatric care", "psychological support",
      "mental health services", "crisis intervention", "depression
```

```
treatment",
      "self-harm prevention", "substance abuse treatment",

      # Crisis Intervention and Support Services
      "suicide prevention hotline", "crisis intervention teams", "emergency
psychiatric services",
      "mobile crisis units", "peer support programs", "safety planning",
      "crisis stabilization units", "24/7 crisis text lines",
      "community mental health centers", "assertive community treatment",
"act teams",

      # Pharmaceuticals for Mental Health
      ## Antidepressants
      # SSRIs
      "fluoxetine", "prozac", "sarafem", "sertraline", "zoloft",
"citalopram", "celexa",
      "escitalopram", "lexapro", "paroxetine", "paxil", "seroxat",
"fluvoxamine", "luvox",
      # SNRIs
      "venlafaxine", "effexor", "duloxetine", "cymbalta", "desvenlafaxine",
"pristiq",
      "levomilnacipran", "fetzima",
      # TCAs
      "amitriptyline", "elavil", "endep", "nortriptyline", "pamelor",
"desipramine", "norpramin",
      "imipramine", "tofranil", "clomipramine", "anafranil", "doxepin",
"sinequan", "trimipramine",
      "surmontil", "protriptyline", "vivactil",
      # MAOIs
      "phenelzine", "nardil", "tranylcypromine", "parnate",
"isocarboxazid", "marplan",
      "selegiline", "emsam",

      ## Atypical Antidepressants
      "bupropion", "wellbutrin", "zyban", "mirtazapine", "remeron",
"nefazodone", "serzone",
      "trazodone", "desyrel", "oleptro", "vilazodone", "viibryd",
"vortioxetine", "trintellix",

      ## Antipsychotics
      # Typical Antipsychotics
      "haloperidol", "haldol", "chlorpromazine", "thorazine",
"fluphenazine", "prolixin",
```

```
      "perphenazine", "trilafon", "thioridazine", "mellaril",
"thiothixene", "navane",
      "loxapine", "loxitane", "pimozide", "orap", "trifluoperazine",
"stelazine",
      # Atypical Antipsychotics
      "risperidone", "risperdal", "olanzapine", "zyprexa", "quetiapine",
"seroquel",
      "aripiprazole", "abilify", "ziprasidone", "geodon", "clozapine",
"clozaril",
      "paliperidone", "invega", "asenapine", "saphris", "lurasidone",
"latuda",
      "iloperidone", "fanapt", "brexpiprazole", "rexulti", "cariprazine",
"vraylar",

      ## Mood Stabilizers
      "lithium", "lithobid", "lamotrigine", "lamictal", "valproate",
"depakote",
      "divalproex sodium", "carbamazepine", "tegretol", "oxcarbazepine",
"trileptal",
      "gabapentin", "neurontin", "topiramate", "topamax",

      ## Anti-Anxiety Medications
      # Benzodiazepines
      "alprazolam", "xanax", "lorazepam", "ativan", "diazepam", "valium",
      "clonazepam", "klonopin", "chlordiazepoxide", "librium",
"clorazepate", "tranxene",
      "oxazepam", "serax", "temazepam", "restoril", "triazolam", "halcion",
      # Non-Benzodiazepine Anxiolytics
      "buspirone", "buspar", "hydroxyzine", "atarax", "vistaril",
"pregabalin", "lyrica",
      "meprobamate", "miltown", "equanil", "propranolol", "inderal",
"prazosin", "minipress",

      ## Stimulants (for ADHD and related conditions)
      "methylphenidate", "ritalin", "concerta", "daytrana", "metadate",
"aptensio",
      "dexmethylphenidate", "focalin", "amphetamine", "adderall",
"mydayis", "evekeo",
      "dextroamphetamine", "dexedrine", "vyvanse", "lisdexamfetamine",
"desoxyn", "methamphetamine",

      ## Sleep Aids
      "zolpidem", "ambien", "eszopiclone", "lunesta", "zaleplon", "sonata",
```

```
      "ramelteon", "rozerem", "suvorexant", "belsomra", "trazodone",
"desyrel",
      "doxepin", "silenor", "mirtazapine", "remeron",

      ## Other Medications
      "atomoxetine", "strattera", "guanfacine", "intuniv", "clonidine",
"kapvay",
      "modafinil", "provigil", "armodafinil", "nuvigil",

      # General Suicide Prevention Terms
      "suicide prevention", "mental health program", "psychiatric care",
      "psychological support", "mental health services", "crisis
intervention",
      "depression treatment", "self-harm prevention", "substance abuse
treatment",

      # Medical Terminology Related to Mental Health
      "electroconvulsive therapy", "ect", "repetitive transcranial magnetic
stimulation",
      "rtms", "ketamine therapy", "psychiatric hospitalization",
"involuntary commitment",
      "partial hospitalization program", "intensive outpatient program",
"iop",
      "behavioral activation", "exposure therapy", "systematic
desensitization",
      "schema therapy", "attachment-based therapy"

      ## Holistic and Alternative Therapies
      "acupuncture", "meditation", "mindfulness", "yoga", "tai chi",
"qigong",
      "herbal medicine", "chinese medicine", "ayurveda", "homeopathy",
"naturopathy",
      "aromatherapy", "massage therapy", "reiki", "energy healing",
"hypnotherapy",
      "chiropractic care", "sound therapy", "music therapy", "art therapy",
      "animal-assisted therapy", "equine therapy", "nature therapy",
"forest bathing",
      "holistic nutrition", "functional medicine", "integrative medicine",
      "breathwork", "progressive muscle relaxation", "guided imagery",
      "biofeedback", "light therapy", "sensory deprivation therapy",
      "float therapy", "spiritual counseling", "faith-based therapy",
      "psychoneuroimmunology", "vagus nerve stimulation", "ayahuasca
therapy",
```

```
      "psilocybin-assisted therapy", "mdma-assisted therapy", "ketamine
therapy",
      "microdosing", "craniosacral therapy", "somatic therapy", "rolfing",
      "feldenkrais method", "alexander technique"
      ]

      df = pd.read_excel(file_path +
r'\v3.scrubbed_data_with_grant_info.xlsx')

      unwanted_texts = [
      "no abstract available.ko",
      "no abstract available",
      "no abstract was provided. however, the first page of this article is
available by following the doi. en",
      "no abstract was provided. however, the first page of this article is
available by following the doi.
https://journals.lww.com/hrpjournal/citation/1999/07000/to_change_the_patie
nt_or_the_patient_s_world__the.4.aspx en",
      "no abstract was provided. however, the first page of this article is
available by following the doi. suicide selfr-harmen",
      "no abstract was provided. however, the first page of this article is
available by following the doi.en",
      "no abstract was provided. however, the first page of this article is
available by following the doi.en"
      ]
      df = df[~df['Abstract'].isin(unwanted_texts)]  # Remove exact matches
      df = df[df['Abstract'].str.strip() != ""]  # Remove completely blank
abstracts
      df = df.dropna(subset=['Abstract'])  # Remove NaN values

      check_removed_abstracts(df, unwanted_texts)

      print(df[['Abstract']].head(10))  # Display the first 10 rows of the
'abstract' column
      print(df['Abstract'].isna().sum())  # Count missing values in
'abstract'

      df[['translated_abstract', 'was_translated']] =
df['Abstract'].astype(str).apply(lambda x:
pd.Series(translate_to_english(x)))

      count_translated_abstracts(df)
```

```
    download_translated_abstracts_csv(df)

    df['cleaned_abstract'] =
df['translated_abstract'].apply(preprocess_text)

    df['tokenized_abstract'] =
df['cleaned_abstract'].apply(tokenize_text)

    download_tokenized_csv(df)

    # Remove duplicate abstracts after tokenization
    df = df.drop_duplicates(subset=['tokenized_abstract'])

    # Save the updated tokenized CSV
    download_tokenized_csv(df)

    # TF-IDF Vectorization
    vectorizer = TfidfVectorizer(max_features=20, stop_words='english')
# Extract top 20 keywords
    tfidf_matrix = vectorizer.fit_transform(df['tokenized_abstract'])

    # Get top keywords
    feature_names = vectorizer.get_feature_names_out()
    tfidf_scores = tfidf_matrix.sum(axis=0).A1  # Sum TF-IDF scores
across all documents

    # Convert to DataFrame
    tfidf_df = pd.DataFrame({'Keyword': feature_names, 'Score':
tfidf_scores})
    tfidf_df = tfidf_df.sort_values(by="Score", ascending=False)

    # Display top keywords
    print(tfidf_df)

    # Save the results to a CSV file for manual review
    tfidf_df.to_csv(dir_path + r'\tfidf_keywords.csv', index=False)
    print("TF-IDF keywords saved as 'tfidf_keywords.csv'")

    # Apply the function to tokenized abstracts
    df['prevention_methods'] =
df['tokenized_abstract'].apply(extract_prevention_methods)

    # Display extracted prevention methods
```

```
      print(df[['Title', 'tokenized_abstract',
'prevention_methods']].head(10))

      # Save extracted prevention methods for manual review
      df[['Title', 'tokenized_abstract',
'prevention_methods']].to_csv(dir_path + r'\prevention_methods.csv',
index=False)
      print("Extracted prevention methods saved as
'prevention_methods.csv'")

      # Apply the regex function to each tokenized abstract
      df['extracted_prevention_methods'] =
df['tokenized_abstract'].apply(lambda x: extract_prevention_methods2(x) if
isinstance(x, str) else None)

      # Display sample results
      print(df[['Title', 'tokenized_abstract',
'extracted_prevention_methods']].head(10))

      # Save extracted methods for review
      df[['Title', 'tokenized_abstract',
'extracted_prevention_methods']].to_csv(dir_path +
r'\extracted_prevention_methods.csv', index=False)
      print("Extracted prevention methods saved as
'extracted_prevention_methods.csv'")

      # Display first few rows of the prevention methods frequency
      all_methods = []
      for methods in df['extracted_prevention_methods'].dropna():
      all_methods.extend([m.strip() for m in methods.split(",")])

      method_counts = Counter(all_methods)
      method_counts_df = pd.DataFrame(method_counts.items(),
columns=['Method', 'Count']).sort_values('Count', ascending=False)

      print(method_counts_df.head(20))

      # Plot top 20 prevention methods
      top_n = 20
      top_methods = method_counts_df.head(top_n)

      plt.figure(figsize=(12, 6))
      plt.barh(top_methods['Method'], top_methods['Count'],
```

```
color='skyblue')
      plt.xlabel("Frequency")
      plt.ylabel("Prevention Method")
      plt.title(f"Top {top_n} Suicide Prevention Methods")
      plt.gca().invert_yaxis()  # Reverse order for readability
      plt.show()


if __name__ == '__main__':
      dir_path = os.path.dirname(os.path.realpath(__file__)) +
r'\safetylit_records'

      main()
```

Appendix 9 Validation:

```
"""
# Suicide Research Dataset Analysis
This notebook performs a clean, professional analysis of the provided
suicide research dataset. The focus is on publication trends, missing
values, and informative visualizations.

---

## 1. Imports and Dataset Loading
"""
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from collections import Counter
import re

def main(file_path):
      # Set aesthetics
      sns.set(style="whitegrid", palette="muted", font_scale=1.2)

      # Load the dataset
      df = pd.read_excel(file_path + r"\v4.complete_data.xlsx")
```

```
      print(f"Dataset contains {df.shape[0]:,} rows and {df.shape[1]:,}
columns.")

      """## 2. Missing Value Analysis
      We'll inspect the dataset for missing values and highlight the top
problematic columns.
      """

      # Columns to inspect
      columns_of_interest = [
      "Combined_Author_Details", "Author_Affiliations", "Journal_Name",
"Publication_Year",
      "Language", "Keywords", "DOI", "Research Methodology", "Data_Source",
      "Grant_Information", "Suicide_Prevention_Measures", "Abstract"
      ]


      # Subset dataframe
      df_copy = df[columns_of_interest].copy()

      # Normalize Grant_Information: treat blank or "none found" as missing
      df_copy['Grant_Information'] =
df_copy['Grant_Information'].astype(str).str.strip().str.lower()
      df_copy['Grant_Information'] =
df_copy['Grant_Information'].replace(['', 'none found'], pd.NA)

      # Normalize Abstract: treat known placeholders as missing
      abstract_placeholders = {
      "<p>[abstract unavailable]</p>",
      "[abstract unavailable]",
      "<p>language: english</p>",
      "<p>[abstract unavailable]</p> <p>language: english</p>",
      "no abstract available.ko",
      "no abstract available",
      "no abstract was provided. however, the first page of this article is
available by following the doi. en",
      "no abstract was provided. however, the first page of this article is
available by following the doi.
https://journals.lww.com/hrpjournal/citation/1999/07000/to_change_the_patie
nt_or_the_patient_s_world__the.4.aspx en",
      "no abstract was provided. however, the first page of this article is
available by following the doi. suicide selfr-harmen",
      "no abstract was provided. however, the first page of this article is
```

```
available by following the doi.en"
      }

      df_copy['Abstract'] =
df_copy['Abstract'].astype(str).str.strip().str.lower()
      df_copy['Abstract'] =
df_copy['Abstract'].replace(abstract_placeholders, pd.NA)
      df_copy['Abstract'] = df_copy['Abstract'].replace('nan', pd.NA)  #
Handle stringified NaNs

      # Compute missing counts and percentages
      total_rows = len(df_copy)
      missing_counts = df_copy.isnull().sum()
      missing_percent = (missing_counts / total_rows * 100).round(1)

      # Prepare DataFrame for plotting
      missing_df = pd.DataFrame({
      'Missing Count': missing_counts,
      'Missing %': missing_percent
      }).sort_values(by='Missing Count', ascending=False)

      # Plot
      plt.figure(figsize=(14, 6))
      ax = sns.barplot(x=missing_df.index, y=missing_df['Missing Count'],
color="steelblue")

      # Add percentage labels
      for i, (count, pct) in enumerate(zip(missing_df['Missing Count'],
missing_df['Missing %'])):
      ax.text(i, count + total_rows * 0.01, f"{pct}%", ha='center',
va='bottom', fontsize=10)

      plt.title("Missing Values in Selected Key Columns", fontsize=14)
      plt.ylabel("Number of Missing Entries")
      plt.xticks(rotation=45, ha='right')
      plt.tight_layout()
      plt.show()

      """## 3. Publication Year Analysis
      Let's look at how research on suicide has evolved over time based on
publication years.
      """
```

```
    # Clean up the publication year
    df['Publication_Year'] = pd.to_numeric(df['Publication_Year'],
errors='coerce')
    df_years = df.dropna(subset=['Publication_Year'])

    plt.figure(figsize=(12, 6))
    sns.histplot(df_years['Publication_Year'], bins=30, kde=True)
    plt.title("Distribution of Publication Years")
    plt.xlabel("Publication Year")
    plt.ylabel("Number of Publications")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

    # Top publication years
    top_years = df_years['Publication_Year'].value_counts().head(10)
    print("Top 10 Most Frequent Publication Years:")
    print(top_years)

    """## 4. Language Distribution
    Examine the distribution of article languages.
    """

    plt.figure(figsize=(10, 5))
    lang_counts = df['Language'].value_counts().head(10)
    sns.barplot(x=lang_counts.index, y=lang_counts.values)
    plt.title("Top 10 Languages Used in Publications")
    plt.xlabel("Language")
    plt.ylabel("Count")
    plt.tight_layout()
    plt.show()

    """## 5. Grant Information Availability
    Check how many papers have associated grant data.
    """

    # Normalize and check grant information
    grant_col =
df['Grant_Information'].fillna('').str.strip().str.lower()
    has_grant_info = ~grant_col.isin(['', 'none found'])

    # Count True (has grant) vs False (no grant)
    grant_info_counts = has_grant_info.value_counts()
```

```
    # Plot
    plt.figure(figsize=(6, 4))
    sns.barplot(
    x=grant_info_counts.index.map({True: 'Has Grant Info', False: 'No
Grant Info'}),
    y=grant_info_counts.values
    )
    plt.title("Availability of Grant Information")
    plt.ylabel("Number of Articles")
    plt.tight_layout()
    plt.show()

    """## 6. Keyword Analysis (If Available)
    A simple frequency chart of the most common keywords.
    """

    if 'Keywords' in df.columns and df['Keywords'].notna().sum() > 0:
    dfk = df[df['Keywords'] != r"['N\\A']"]
    all_keywords = dfk['Keywords'].dropna().str.lower().str.split(', ')
    flat_keywords = []
    for sublist in all_keywords:
        for keyword in sublist:
            # Remove all punctuation except / and &
            cleaned = re.sub(r"[^\w\s/&]", "", keyword.strip())
            # Remove any remaining single quotes that might be left
from original format
            cleaned = cleaned.replace("'", "")
            flat_keywords.append(cleaned)
    keyword_counts = Counter(flat_keywords)
    common_keywords = keyword_counts.most_common(20)

    keywords_df = pd.DataFrame(common_keywords, columns=["Keyword",
"Count"])

    plt.figure(figsize=(12, 6))
    sns.barplot(data=keywords_df, x="Keyword", y="Count")
    plt.xticks(rotation=45)
    plt.title("Top 20 Most Common Keywords")
    plt.tight_layout()
    plt.show()

    nankeys = df[df['Keywords'] == r"['N\\A']"]
```

```
    nokeys = df[df['Keywords'].isna()]
    kwmissing = pd.concat([nankeys, nokeys])
    print(f'Percent Missing Keywords: {100 *
(kwmissing.shape[0]/df.shape[0])}')
    print(f'Number missing keywords: {kwmissing.shape[0]}')

    """## 7. Save Cleaned DataFrame (Optional)"""

    # df.to_csv("cleaned_suicide_research_data.csv", index=False)
    # print("Cleaned data saved.")


if __name__ == '__main__':
    file_path = os.path.dirname(os.path.realpath(__file__)) +
r'\safetylit_records'
    main(file_path)
```